

Quantum random access memory: a survey and critique

Samuel Jaques^{1,2} and Arthur G. Rattew³

¹Department of Combinatorics and Optimization, University of Waterloo, 200 University Ave W, Waterloo, Canada

²Institute for Quantum computing, University of Waterloo, 200 University Ave W, Waterloo Canada

³Department of Materials, University of Oxford, Parks Road, Oxford OX1 3PH, United Kingdom

Quantum random-access memory (QRAM) is a mechanism to access data (quantum or classical) based on addresses which are themselves a quantum state. QRAM has a long and controversial history, and here we survey and expand arguments and constructions for and against.

We use two primary categories of QRAM from the literature: (1) active, which requires external intervention and control for each QRAM query (e.g. the error-corrected circuit model), and (2) passive, which requires no external input or energy once the query is initiated. In the active model, there is a powerful opportunity cost argument: in many applications, one could repurpose the control hardware for the qubits in the QRAM (or the qubits themselves) to run an extremely parallel classical algorithm to achieve the same results just as fast. We apply these arguments in detail to quantum linear algebra and prove that most asymptotic quantum advantage disappears with active QRAM systems, with some nuance related to the architectural assumptions.

Escaping the constraints of active QRAM requires ballistic computation with passive memory, which creates an array of dubious physical assumptions, which we examine in detail. Considering these details, in everything we could find, all non-circuit QRAM proposals fall short in one aspect or another.

In summary, we conclude that *cheap, asymptotically scalable* passive QRAM is unlikely with existing proposals, due to fundamental obstacles that we highlight.

Samuel Jaques: sejaques@uwaterloo.ca

Arthur G. Rattew: arthur.rattew@materials.ox.ac.uk

These obstacles are deeply rooted in the requirements of QRAM, but are not provably inevitable; we hope that our results will help guide research into QRAM technologies that circumvent or mitigate these obstacles. Finally, circuit-based QRAM still helps in many applications, and so we additionally provide a survey of state-of-the-art techniques as a resource for algorithm designers using QRAM.

1 Introduction

Computers are machines to process data, and so access to data is a critical function. Quantum computers are no exception. Specialized memory-access devices (e.g., RAM) are now commonplace in classical computers because they are so useful. It thus seems intuitive that future quantum computers will have similar devices.

Unfortunately for quantum computers, it's not clear that the analogy holds. Memory access, classical or quantum, requires a number of gates that grows proportional to the memory size. For a classical computer, manufacturing gates is generally a fixed cost and we care more about the runtime (though this assumption starts to break down with large scale, high-performance computing). In contrast, gates in most quantum technologies are active interventions, requiring energy or computational power to enact.

The foundational work on QRAM, [52], attempts to break from this model and imagines passive components to enact the memory, such that after a signal is sent into the device, it propagates without external control (i.e. ballistically) to complete the memory access. This would be more efficient, but might not be realistic.

QRAM has sparked significant controversy and yet numerous applications presume its existence. Because of this, results are scattered. Circuit-

based QRAM, where one accepts a high gate cost, can be still be useful, so optimized techniques appear as intermediate results in subject-specific works. Here we attempt to survey and collect all such results.

Similarly, there is a broad literature of attempts to construct QRAM (in theory or experiment) and many published criticisms of it. This paper aims to unify the existing criticisms and provide new ones, and consider all existing QRAM proposals in light of these criticisms.

1.1 Summary

To frame this paper, we first summarize existing arguments for QRAM and try to explain, conceptually, how to think about the nature of a QRAM device.

Classically we are used to “gates” being static physical components that data propagates through, just as depicted in circuit diagrams. Applying that intuition to quantum computing is unjustified. In contrast, the memory peripheral framework of [63] models quantum computing like Figure 1, where data (in the form of qubits) are static physical components, and gates are operations that a controller applies to the data. In this framework a quantum computer is a physical object (e.g., some Hilbert space) called a *memory peripheral* that evolves independently under some Hamiltonian. There is also a *memory controller* that can choose to intervene on the object, either by applying a quantum channel from some defined set, or modifying the Hamiltonian. This more accurately reflects today’s superconducting and trapped-ion quantum computers, as well as surface code architectures of the future where classical co-processors will be necessary to handle error syndrome data, possibly in addition to scheduling and applying gates.

While future devices may drastically reduce the overhead of each operation, there will still be some level of computational power necessary to manage each intervention in the device. As this paper will focus primarily on asymptotics, this distinction will matter less; e.g., a laser’s micro-controller is not much different than any other application-specific integrated circuit.

Overall, this framework highlights a view of a quantum computer as a peripheral of a classical device. We imagine a quantum computer as a collection of quantum devices with many associ-

ated classical computers built into the architecture whose purpose is to manage the quantum devices.

The crux of QRAM is that if a query state is a superposition over all addresses, then for the device or circuit to respond appropriately, it must perform a memory access over *all* addresses simultaneously. While “it performs all possible computations at once” is a classic misunderstanding of quantum computing, it applies more directly to QRAM: imagining the memory laid out in space, a QRAM access must transfer some information to each of the bits in memory if it hopes to correctly perform a superposition of memory accesses. This sets up an immediate contrast to classical memory, which can instead adaptively direct a signal through different parts of a memory circuit.

In the memory peripheral framework, we need to apply gates for every possible pattern of memory access, and since each gate requires some action from the controller – be it time, computation, or energy – the relevant cost is proportional to the total size of the memory. The problems with high time or energy costs are obvious, but Figure 1 highlights that a computational cost is really an *opportunity* cost. If we have a dedicated co-processor for every $O(1)$ qubits, could they do something besides run the quantum computer?

This captures arguments against particular uses of QRAM. If we are allowed to reprogram the control circuitry, then:

- two-to-one collision finding is just as fast for the quantum computer or its controller [16]
- claw finding is *faster* for the controller than the quantum computer [63]
- the quantum advantage for matrix inversion drops from exponential to (at best) polynomial according to [1, 39] (see Section 4 for more nuance)

In short, a massive quantum computer implies a massive classical co-processor to control it, and that co-processor can quickly solve many problems on its own.

This holds for what [39] calls “active” QRAM. Proponents of QRAM can then argue that this is the wrong way to conceptualize QRAM. In classical computers, RAM is a separate device, manufactured differently than the CPU, designed

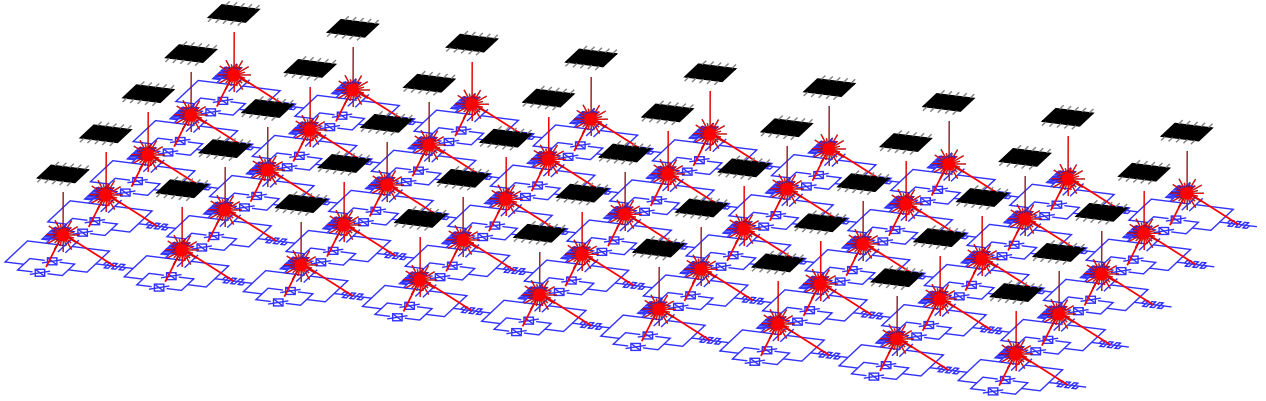


Figure 1: An illustration of a quantum computer in the memory peripheral model [63]. Classical controllers are the chips on the top, which send signals (in red) to qubits (in blue).

specifically to perform memory accesses. From the CPU’s perspective, it sends a signal into the RAM and waits for the RAM to process that signal on its own. An analogous quantum device – what [1] denotes as “passive” QRAM – would not incur the costs of external control, but would need incredible engineering to function correctly without that control.

The memory peripheral framework also allows more complicated Hamiltonians and longer evolution to capture this situation. This reflects “ballistic” computation, where some environment is prepared and then allowed to propagate on its own. Today’s boson sampling experiments are more like this: the classical control (e.g. motors or humans) must establish the optical layout and then inject a photon, but after this setup, computation will proceed without any further intervention.

The main technical contributions of this paper are a careful analysis of the requirements and assumptions of this other perspective. In short, we argue that for passive QRAM:

- If the QRAM requires no intervention, it is “ballistic”, and must be described by a single time-independent Hamiltonian;
- No intervention means no active error correction, and hence each physical component must intrinsically have near-zero noise levels;
- Unless the device is carefully designed to limit error propagation, the physical component error rates must be problematically low;

- To be compatible with the rest of a fault tolerant computation, we need some way to apply the QRAM, which (in the passive case) acts only on physical qubits, to the logical qubits in an error corrected code;
- In particular, a QRAM gate cannot be teleported, so we must resort to other techniques which risk decohering the state.

In many ways the comparison to classical RAM is inaccurate, and passive QRAM faces daunting engineering challenges. In our opinion, these challenges are probably insurmountable, in which case quantum memory would always need to be active, and thus access to N bits of QRAM would always have a total (parallelizable) cost proportional to N or higher.

We stress that QRAM can still be a useful tool: in numerous applications in chemistry and quantum arithmetic, such as those in Section 3, QRAM can offload expensive computations to a classical co-processor, providing a benefit that exceeds the overall access cost.

1.2 Outline

We first define our models of quantum computation and the definitions of QRAM in Section 2. There we summarize the different names and notions of this device from the literature, to unify the discussion.

Section 3 discusses the main uses of QRAM, and the requirements that each application places on the QRAM device, and the scale they require.

Section 4 analyses the implications of various QRAM models for the application of quantum linear algebra. We prove that in many regimes,

an active QRAM cannot have quantum advantage over a parallel classical algorithm for a certain general linear algebra tasks.

Then in Section 5 we give state-of-the-art examples of *circuit* QRAM from the literature. These are circuits to perform QRAM that are intended to work in the fault-tolerant layer by building the QRAM gate out of a more fundamental gate set. As expected, the cost is proportional to the number of bits of memory, and we show a tight lower bound for this.

Section 6 then explores the concept of passive hardware-QRAM, i.e., a device tailor-made to perform QRAM accesses without intervention, with Section 7 focusing specifically on errors. These describe the precise errors listed in the introduction.

While some of these issues are quantum-specific, others seem to also apply to classical RAM, which we know is a feasible technology. In Section 8, we apply the same arguments to classical RAM. For each argument, either it does not apply because classical memory and its access is an easier task, or the analysis and real-world use of memory in large-scale devices already accounts for the costs we describe.

We survey approaches to the bucket-brigade in Section 9, noting how each different proposal fails in one or more of the criteria above. Section 10 describes 3 other architectures and one error correction method, all of which also fall short.

While Section 2 and Section 3 will be useful for all readers, those interested in *using* QRAM in their algorithms should focus on Section 5. Section 4 is intended for readers focusing on quantum linear algebra. Those interested in the arguments against QRAM should read Section 6, Section 7, and Section 8. Readers already familiar with this debate might focus on the original results, such as Theorem 7.1, the critiques of specific schemes in Section 9 and Section 10, and the analysis of quantum linear algebra in Section 4.

Contents

1	Introduction	1
1.1	Summary	2
1.2	Outline	3
2	Definitions and Notation	5
2.1	QRAM Definitions	5

2.2	Routing and Readout	6
2.3	Alternate Notation	7
3	Applications	7
4	Case Study: Quantum Linear Algebra	9
4.1	Input Model and Matrix Functions	9
4.2	Dequantization	10
4.3	QRAM versus Parallel Classical Computation	11
4.4	Quantum Linear Algebra with Noisy QRAM	13
5	Circuit QRAM	14
5.1	Lower Bounds	14
5.2	Unary Encoding	16
5.3	Bucket-Brigade	17
5.4	Select-Swap	18
5.5	Parallel QRAM	19
5.6	Data Structures	20
6	Hardware QRAM	20
6.1	Active vs. Passive QRAM	21
6.2	Path Independence	21
6.3	Hamiltonian Construction	22
7	Errors	23
7.1	Error-corrected components	23
7.2	Error Correction	24
7.3	Gate Teleportation	24
8	Classical RAM	26
8.1	Area-Time Metric	26
8.2	Active Components	27
8.3	Error Propagation and Correction	29
8.4	Latency	29
9	Bucket-Brigade QRAM	30
9.1	Construction	30
9.2	Reversibility	30
9.3	Proposed Technologies	31
9.4	Error Rates	32
10	Other Proposals	33
10.1	Time-bins	34
10.2	Quantum Optical Fanout	35
10.3	Phase Gate Fanout	36
10.4	Derangement Codes	37
10.5	Two-Level Quantum Walkers	37
11	Discussion	38
	References	39

A Linear Algebra Proofs	46
B Proof of Theorem 5.1	48
C Proof of No Distillation	49

2 Definitions and Notation

We will frequently use asymptotic notation. Writing $f(n) \in O(g(n))$ means that $g(n)$ is an asymptotic upper bound for f , $f(n) \in \Omega(g(n))$ means $g(n)$ is an asymptotic lower bound, and $f(n) \in \Theta(g(n))$ means both. $f(n) \in o(g(n))$ means that $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 0$. A tilde over any of these means we are ignoring logarithmic factors, e.g., $\tilde{O}(g(n))$ means $O(g(n) \log^k(n))$ for some constant k . We may write $f(n) = O(g(n))$ to mean the same as $f(n) \in O(g(n))$.

2.1 QRAM Definitions

We will follow the notation of Kuperberg [72], who proposes a taxonomy of four types of memory, based on the product of two criteria:

- Does the device store classical or quantum data? (CM for classical memory or QM for quantum memory)
- Can the device access this data in superposition or not? (CRA for classical access or QRA for quantum access)

The “R” stands for “Random”, though we will ignore this because a memory device does not need to gracefully handle random access patterns to fit the definitions.

This results in four types of memory:

CRACM (Classical access classical memory): This is classical memory that can only be accessed classically, i.e., the memory we are all familiar with today. Kuperberg does not actually distinguish the *speed* of memory access. Comparing quantum and classical, we do not really care. In this sense DRAM, flash memory, hard disk drives, and even magnetic tapes can all be considered CRACM. We discuss more properties of CRACM in Section 8.

CRAQM (Classical access quantum memory): Here we have quantum memory, but which can only be accessed classically. The notion of “access” is somewhat fuzzy: obviously a classical controller cannot read quantum memory without destructively measuring it. Our notion of access will be more akin to “addressable”, meaning the classical computer can find the required bits of quantum memory. This means essentially all qubits are CRAQM, since any quantum circuit will require a classical controller to find the necessary qubits to apply the gates for that circuit.

Classically addressing qubits is a challenging and extremely important problem (e.g., [64]), but it is fundamentally different and mostly unrelated to what this paper addresses.

QRACM (Quantum access classical memory): We formally define “QRACM” as follows:

Definition 1. *Quantum random-access classical memory (QRACM) is a collection of unitarities $U_{QRACM}(T)$, where $T \in \{0,1\}^N$ is a table of data, such that for all states $|i\rangle$ in the computational basis, $0 \leq i \leq N-1$,*

$$U_{QRACM}(T) |i\rangle |0\rangle = |i\rangle |T_i\rangle. \quad (1)$$

Here T is the classical memory and $|i\rangle$ is an address input. We call the register with $|i\rangle$ the *address register* and the bit which transforms from $|0\rangle$ to $|T_i\rangle$ the *output register* (recall $T_i \in \{0,1\}$). Since $U_{QRACM}(T)$ is unitary, superposition access follows by linearity. Our definition specifies a unitary action, but we do not require the device to be unitary: a quantum channel whose action is identical to (or approximates) conjugation by $U_{QRACM}(T)$ will still be considered QRACM (for example, the method in [19] uses measurement-based uncomputation).

In this definition, the table T parameterizes the unitaries, since QRACM must take some classical data as input. This means that there is no single QRACM “gate”: there is a *family* of gates, parameterized by T . In fact, since T has N entries of $\{0,1\}$, there are 2^N possible memories.

In some literature the table is written as a function f so that $T_i = f(i)$. In this case, the QRAM might be written as an oracle O_f implementing the mapping $O_f |x\rangle |0\rangle = |x\rangle |f(x)\rangle$. For some QRACM methods, the table does not need to be in classical memory if f is efficiently computable.

We take N as the number of elements in the table that the QRAM must access, which we will sometimes call *words* (for cases where they are larger than single bits). This means the address register has $n := \lceil \lg N \rceil$ qubits. Other papers choose a notation focusing on n , the number of qubits in the address, so that the number of words in the table is 2^n (and the number of possible tables is 2^{2^n}). Using n and 2^n emphasizes that, for certain algorithms, the size of the table will be exponential compared to other aspects of the input and running time. We will mainly use N and $\lceil \lg N \rceil$ since the classical table must be input to the algorithm, and therefore anything $O(N)$ is linear in the input size, but we use n throughout as shorthand for $\lceil \lg N \rceil$.

Of course most applications require the elements of T to be more than single bits, but concatenating single-qubit QRAM gives multi-bit QRAM.

QRAQM (Quantum access quantum memory) : We formally define this as:

Definition 2. *Quantum random-access quantum memory (QRAQM) is a collection of unitaries $U_{QRAQM}(N)$, where $N \in \mathbb{N}$, such that for all states $|i\rangle$ with $0 \leq i \leq N-1$ and all N -qubit states $|T_0, T_1, \dots, T_{N-1}\rangle$ (referred to as the data register, where $T_0, \dots, T_{N-1}$ is a bitstring),*

$$U_{QRAQM} |i\rangle |0\rangle |T_0, \dots, T_{N-1}\rangle = |i\rangle |T_i\rangle |T_0, \dots, T_{N-1}\rangle \quad (2)$$

The difference now is that the memory itself is a quantum state, meaning we could have a superposition of different tables. Here the gate is implicitly parameterized by the table size, and needs to suffice for all possible states of that size.

This definition only shows a unitary that can *read* the quantum memory, but one can show that unitarity forces

$$U_{QRAQM} |i\rangle |1\rangle |T_0, \dots, T_{N-1}\rangle = |i\rangle |1 \oplus T_i\rangle |T_0, \dots, T_{N-1}\rangle \quad (3)$$

and from this, one can conjugate $U_{QRAQM}(N)$ with Hadamard gates on all qubits in the output and data registers to construct a write operation.

The literature still has some ambiguity about this definition. For example, we could define a

SWAP-QRAQM such that

$$\begin{aligned} U_{S-QRAQM} |i\rangle |\psi\rangle |\phi_0\rangle \dots |\phi_{N-1}\rangle \\ = |i\rangle |\phi_i\rangle |\phi_0\rangle \dots |\phi_{i-1}\rangle |\psi\rangle |\phi_{i+1}\rangle |\phi_{N-1}\rangle. \end{aligned} \quad (4)$$

SWAP-QRAQM is equivalent to the previous read-and-write QRAQM, if some ancilla qubits and local gates are used.

2.2 Routing and Readout

Imprecisely, QRAM and QRAQM have two tasks: the first is routing and the second is readout.

By analogy, consider retrieving data classically from a network database. A request for the data will contain some address, and this will need to be appropriately routed to a physical location, such as a specific sector on a specific drive in a specific building. Once the signal reaches that location, the data must be copied out before it can be returned. For example, this could involve reading the magnetization of the hard drive.

Difficulties can arise from either routing or readout, but we stress that both are necessary. [72] states “Our own suggestion for a QRAM architecture is to express classical data with a 2-dimensional grid of pixels that rotate the polarization of light. (A liquid crystal display has a layer that does exactly that.) When a photon passes through such a grid, its polarization qubit reads the pixel grid in superposition.” This seems to satisfy the requirements of the readout, but does not address routing: how do we generate the photon in such a way that its direction is precisely entangled with the address register, so that for state $|i\rangle$, the photon travels to the i th pixel in this grid?

Generally, the routing problem seems more difficult. If we have a device that routes data such that the physical location of some aspect of the device (such as the photon described above) becomes entangled with the input state given as address, then readout becomes relatively simple. A readout with the polarization mechanism described above could give QRAM, or a series of parallel controlled SWAP gates could give QRAQM. Thus, we mainly focus on routing in this paper.

For that reason, we also will use the term “QRAM” to refer to a routing technology that

would work for either QRACM or QRAQM with only modest changes to the readout mechanism (the bucket-brigade routing from [60] is a perfect example).

2.3 Alternate Notation

“QRAM” is used inconsistently throughout the literature. It is sometimes used for QRAQM or QRACM (e.g. [94] uses “QRAM” for both). It is also sometimes meant to specifically refer to an operation that implements QRACM, but via a single gate.

“QROM” has been used to refer to QRACM implemented as a circuit (e.g., from Clifford+T gates) (e.g., [8]). QRACM must be read-only, so using QRAM to refer to QRAQM and QROM to refer to QRACM would be a consistent alternative notation; however, since the literature uses QRAM to refer to both kinds, we opt against this.

In classical computing, devices such as hard drives or tapes *can* implement the functionality of random-access memory, though the access times are longer and more variable than devices such as DRAM or flash memory. Analogously, different proposals achieve the same required function of QRACM – to access memory in superposition – but with radically different costs and access times.

To be more precise than this, we will use *circuit*-QRAM to refer to proposals to construct superposition access using a standard gate set, especially Clifford+T.

We will use *hardware*-QRAM to refer to proposals for QRAM consisting of a specialized device. Following [1, 39], we further categorize into *active* hardware-QRAM, which requires $\Omega(N)$ active interventions from the classical controller for each access, and *passive* hardware-QRAM, which requires $o(N)$ active interventions.

If we imagine memory access by routing an address through a binary tree, the tree must have $N - 1$ nodes (if N is a power of 2), and each node must do some computation on the incoming signal. The difference between active and passive hardware-QRAM is whether that computation requires some external process to make it happen, or whether it proceeds on its own with no external intervention.

Related Work. The concurrent survey in [94] explains some approaches to QRAQM and QRACM, though they do not consider the physical costs in the same way as our work. Specifically, they claim classical RAM requires $O(2^n)$ “gate activations” and quantum RAM requires $O(n)$, without defining a notion of “gate activations”. Arguably the real situation is reversed: when each gate in a QRAM circuit requires external intervention then it costs $O(2^n)$, and classical RAM only needs to dissipate energy from $O(n)$ gates (see Section 8 for a more nuanced picture).

State preparation is a slight generalization of QRAM, with a circuit that performs

$$|x\rangle |0\rangle \mapsto |x\rangle |\psi_x\rangle \quad (5)$$

for some collection of states $|\psi_x\rangle$ for $x \in \{0, \dots, N - 1\}$. Typically we assume a unitary U_x for each x , such that $U_x |0\rangle = |\psi_x\rangle$. QRACM is the special case where $|\psi_x\rangle = |f(x)\rangle$ (e.g. a specific computational basis state).

The techniques for state preparation and QRAM are generally the same, e.g. the techniques of [114] are similar to bucket-brigade QRAM (Section 5.3) and [104] is like a select-swap QRAM (Section 5.4) using phase rotations.

3 Applications

Mainly as motivation for later discussions, we note some leading applications of QRAM. A key point for all of these applications is that the competing classical algorithms parallelize almost perfectly, and hence the active QRAM opportunity cost arguments apply fully.

Optimizing calculations. Many quantum algorithms perform complicated classical functions in superposition, such as modular exponentiation in Shor’s algorithm or inverse square roots in chemistry problems. Recent techniques [8, 12, 19, 45, 47, 57] choose instead to classically precompute the result for a large portion of inputs, and then use QRACM to look up the result.

The scale of QRACM here is around 2^{15} to 2^{25} bits of memory. Further, the overall error rate can afford to be fairly high. For example, using windowed elliptic curve point addition in Shor’s algorithm requires only about 17 applications of the QRACM gate, so even if each one has much

higher error than other fundamental gates, the overall algorithm will still succeed.

Systematic QRACM errors are also less important in this context, as the inputs are generally large superpositions, so mistakes in a small proportion of inputs will only affect a small fraction of the superposition.

The referenced works already assign an $O(N)$ cost for QRACM look-ups, meaning that the arguments against cheap QRACM in this paper have no effect on their conclusions.

Cheap QRACM would also not drastically reduce the costs. First, for factoring, each table is distinct. Since the classical controller requires at least N operations to construct a table of N words, asymptotically the cost of using QRACM in this context is already proportional to N just to *construct* each QRACM table. While this classical pre-processing is likely much cheaper than a T-gate, it limits how much computation can be offset.

In contrast, quantum chemistry that uses QRACM to look up the value of a function of Coulomb potential [19] will only have to access one table. Thus, cheap QRACM would reduce the cost of these algorithms, but it would only be a polynomial factor.

Dihedral Hidden Subgroup. Kuperberg’s algorithm for solving the dihedral hidden subgroup problem relies on QRACM [72]. In this application, the QRACM table is only read once by the quantum computer. Hence, it is asymptotically irrelevant whether access costs $\Omega(N)$ for the quantum computer or less, since the classical controller has an $\Omega(N)$ cost to construct the table.

For cryptographic applications, the size of each QRACM access ranges from about 2^{18} bits to 2^{51} , depending on the parameters [23, 34, 92]. The robustness to noise is unknown.

Exponential Quantum Cryptanalysis. Algorithms with exponential run-time or space requirements are used in the security analysis for cryptography intended to be quantum-safe. Specifically, QRACM is needed in algorithms in collision finding [25] and lattice sieving [2, 61, 73, 74], and QRAQM is necessary for attacks based on quantum random walks, such as claw-finding [107], information set decoding [66, 70],

subset-sum [18], and other lattice sieving methods [24, 33]. As NIST selected lattice cryptography for standardization in 2022 [84], lattice sieving is critical.

For the asymptotically “fastest” quantum lattice sieving against the smallest proposed parameters of the post-quantum schemes to be standardized [24], the QRACM needs 2^{49} bits. Against mid-scale parameters one needs 2^{78} bits. For collision search against the ubiquitous hash function SHA-256, one needs 2^{93} bits of QRACM.

In addition to the hefty memory requirements, the accuracy of the memory must be extraordinarily high. [98] and [5] show that Grover’s algorithm needs exponentially small QRAQM error, but one can also see that most of these cryptography attacks require finding a *single* element of the memory. If that element is destroyed because of noise, the algorithm fails.

This is the most extreme application of QRAM with the most stringent requirements on the device. We will refer to these algorithms somewhat imprecisely as *database QAA*, to indicate that we are using QAA (quantum amplitude amplification) [26], or a quantum random walk, to search over some classical database.

Context There may be a disconnect in the discourse around the feasibility of QRAM. Proponents and hardware designers may only be considering small-scale regimes of up to 2^{30} bits of QRAM, in which case the arguments presented here and elsewhere *may* be surmountable: classical computational components in 2004 had error rates of 2^{-30} to 2^{-40} [108], so perhaps quantum components could reach the same level.

However, at the scale of e.g. 2^{20} bits, our conclusions are also less relevant; we know (Section 5) how to perform N -bit QRAM with a circuit of $O(N)$ gates, so any algorithm using 2^n bits of QRAM will face only a 2^n overhead if physical QRAM assumptions are wrong. For something like 2^{20} bits, this does not make a large difference, especially considering the extreme uncertainties in future quantum computing overheads and architectures.

In contrast, at the scale of 2^{50} bits, cheap passive QRAM causes a drastic change in algorithm costs, which has incentivized the research into algorithms that use QRAM. Yet, even minute physical costs become relevant if they scale with

memory size. For perspective, if we lose a single visible photon’s worth of energy for every bit in memory for each access, accessing 2^{50} bits costs about 1 kJ.

As we move into cryptographic scales, QRAM becomes clearly absurd. A single photon’s worth of energy per bit means 1 *giga*-Joule per access if we have 2^{80} bits. Even more extreme: in a Fabry-Perot interferometer, the momentum change from a single microwave photon on a 10 g mirror would shift the mirror’s position by a portion of 2^{-98} of the cavity’s length. If the memory needs 2^{-128} precision (as one proposed QRAM technology, see Section 10.3), then this shift is a billion times larger than the precision needed.

Most of our conclusions in this paper suggest that an N -bit QRAM access will need energy/computation/some other cost to scale as $\Omega(N)$ (i.e., be active), and/or each physical component will need error rates of $O(1/N)$. These are asymptotic claims, and it’s beyond what we can say today to claim that at small-scale memory sizes, these terms will dominate other more relevant costs. However, at machine learning scales and beyond, we must consider these factors.

4 Case Study: Quantum Linear Algebra

Certain statistical and/or machine learning problems can be framed as linear algebra, where we transform some vectors by matrices. Quantum linear algebra aims to accelerate these transformations; however, if the input data are large databases, this requires QRAM. The following references provide an overview of some of the techniques used in, and applications of, quantum machine learning, in some cases discussing their assumptions regarding the necessity of QRAM [21, 49, 77, 95]. Moreover, the following references provide an overview of, and detail important techniques in, quantum linear algebra with some discussing the creation of the QRAM data-structures relevant for linear algebra [49, 60, 69, 76, 78, 80, 97].

We now detail our input model and formally define the general linear algebra task we consider. We then discuss the necessity of QRAM (i.e. for “unstructured” input data), and evaluate the opportunity cost of QRAM by giving algorithms for the classical co-processors to solve

the defined linear algebra problem. We conclude that without passive hardware-QRAM, quantum linear algebra algorithms provide no asymptotic advantage, unless we are at a scale large enough that the classical co-processors cannot share access to the same memory, but small enough that signal latency is negligible.

Ref [1] made these opportunity cost arguments for matrix inversion, [103] applied them to an approach to “Quantum PageRank”, and [39] briefly notes that these arguments generalize. This section expands and generalizes these ideas, incorporating questions of physical layout.

4.1 Input Model and Matrix Functions

Formally, we assume we are given some matrix $H \in \mathbb{C}^{N \times N}$ such that $H = H^\dagger$, via query access to the elements of H , and some vector $\mathbf{v} \in \mathbb{C}^N$ again via query access to its elements. The query access is provided via oracles which return the output associated with any given input. For example, the oracle for matrix H may be given by

$$O_H |x\rangle |y\rangle |0\rangle = |x\rangle |y\rangle |H_{xy}\rangle. \quad (6)$$

In the case where H is sparse (i.e. H has at most d non-zero elements in any given row or column), we also assume we are given another oracle specifying the locations of the non-zero elements. Moreover, we restrict our attention to the case where the matrix under consideration is both square and Hermitian, noting that non-square and non-Hermitian matrices $\tilde{H}$ can be embedded into a Hermitian matrix $H := \begin{pmatrix} 0 & \tilde{H} \\ \tilde{H}^\dagger & 0 \end{pmatrix}$, as is commonly done in the literature [60]. In this embedding, the eigenvalues of H are directly related to the $\pm$ singular values of $\tilde{H}$, and its eigenvectors are a simple function of the right and left singular vectors of H . Finally, we define a function f of a Hermitian matrix H as a function of its eigenvalues, i.e. when $H |\lambda_i\rangle = \lambda_i |\lambda_i\rangle$, $f(H) := \sum_j f(\lambda_j) |\lambda_j\rangle \langle \lambda_j|$. This differs slightly from some definitions, which define the function on the singular values. We restrict our attention to general set of linear algebra problems which are equivalent to computing the vector $f(H)\mathbf{v}$. We further restrict f to be a degree- k polynomial, with small loss of generality since polynomials can approximate most functions of interest on the domain $[-1, 1]$. We give the formal problem statement in Problem 1.

Problem 1 (Polynomial Eigenvalue Transform Problem). *Given a polynomial function $f : [-1, 1] \mapsto \mathbb{C}$ of degree at most k , such that $\forall x, |f(x)| \leq 1$, an $N \times 1$ initial vector $\mathbf{v}$ and a d -sparse, $N \times N$, Hermitian matrix H , specified by an oracle O_H (and an additional location-oracle in the case that $d \in o(N)$), such that $\|H\|_2 \leq 1$, compute $f(H)\mathbf{v} / \|f(H)\mathbf{v}\|_2$.*

Many quantum linear algebra tasks can be cast in the framework of Problem 1, such as matrix inversion, Gibbs sampling, power iteration, and Hamiltonian simulation [80]. In essence we are describing the Quantum Singular Value Transform (QSVT) framework [49], which can be seen as a unifying framework for nearly all quantum algorithms as summarized in [80]. The QSVT is defined on the singular values rather than the eigenvalues, but in the case of Hermitian matrices, the singular values are just the absolute eigenvalues, and so the problems are practically equivalent (with some additional tedium).

Of course, QRAM is not always necessary to offer query access to H or $\mathbf{v}$. In some cases, there exist circuits with $\text{polylog}(N)$ depth which can implement the mappings, such as in [20, 96, 110, 115]. With entries that are not efficiently computable, an oracle to H needs QRAM access to the Nd non-zero entries of H . We assume a “wide” QRAM such as a bucket-brigade or fanout-and-swap, which need Nd quantum registers but provide $O(\log(Nd))$ circuit-depth per access. Our analysis holds if the QRAM is implemented with fewer quantum registers but with longer access times, as that benefits the classical co-processors in our comparison.

4.2 Dequantization

Tang [106] first introduced dequantization proving that the quantum algorithm for recommendation systems of Kerenidis and Prakash [69] does not have exponential quantum advantage over a randomized classical algorithm making a similar input assumption. The argument starts by noting that the Nd non-zero elements of the input matrix must be stored somewhere (e.g., by preparing a QRAM gate or data structure), and this creates a $\Omega(dN)$ set-up cost. For example, the nodes of a bucket-brigade QRAM must be initialized with the appropriate data.

A similar amount of classical precomputation

can create a data structure that allows efficient ℓ_2 -norm sampling from all columns of the input matrix. Even if data is added in an on-line setting to the quantum memory (amortizing the cost), with similar cost a classical data structure can also be maintained with online updates. Combined with a large body of classical randomized numerical linear algebra techniques (e.g. [36, 68]), one can obtain classical algorithms for Problem 1 with polylogarithmic dependence on the dimensions of the input.

Dequantization has some limitations. Most approaches require either the matrix to be low-rank [11, 48, 106], or with some caveats, for the matrix to be sparse and the polynomial function being applied to have at most a constant degree [11]. If any classical algorithm could implement matrix inversion on sparse matrices (which is a special case of QSVT) with a polynomial dependence on the condition number (i.e., without a meaningful rank constraint), then since matrix inversion was shown to be BQP-complete [60], such a classical algorithm could efficiently simulate any polynomial time quantum algorithm. This would obviously be an extremely surprising result, and so we cannot expect a classical dequantized QSVT-styled algorithm to have rank-independence, in general. Moreover, dequantization also leaves room for some polynomial quantum advantage, with QSVT requiring $O(k)$ QRAM queries for a degree- k polynomial, compared to an $O(k^9)$ cost for leading classical techniques [11]. In general, see Figure 1 of [37] to compare complexities of dequantization. New classical techniques could close these gaps, but currently there are many algorithms with at least a quartic speed-up over their dequantized counterparts, so if cheap QRAM existed, even early fault-tolerant devices might see an advantage [10].

Critically, dequantization arguments allow cheap *access* to the QRAM, and only consider the opportunity cost of the quantum algorithm’s input assumptions (e.g. the cost to construct the QRAM data structure). Instead we focus on the access cost. If the QRAM requires classical control or co-processing (e.g. laser pulses enacting the gates, or classical resources dedicated to error-correction), these polynomial quantum advantages over their dequantized counterparts vanish (and indeed the quantum algorithms

would potentially even be exponentially slower than their dequantized counterparts) – such arguments even apply for some algorithms that cannot be dequantized.

4.3 QRAM versus Parallel Classical Computation

In this section we discuss the possibility of a general asymptotic quantum speedup in Problem 1, by considering the opportunity cost of the classical control for the QRAM. We assumed the QRAM has $O(Nd)$ logical qubits. With active error correction for each qubit, syndrome measurements and corrections will require substantial classical co-processing, with $O(1)$ classical processors per logical qubit. We assume each of these processors has constant-sized local memory with fast and efficient access. It seems unlikely that any *error-corrected* QRAM architecture could use *asymptotically* fewer classical resources.

Even without error correction, only passive hardware-QRAM avoids costs proportional to Nd . Advances in quantum computing could drive the constant of proportionality to be smaller, but asymptotically the arguments in this section will still hold, short of a significant breakthrough in the manufacturing of passive QRAM (see Section 6 for a comparison of active vs. passive QRAM).

We then further consider some frequently neglected constraints on large scale parallel computing, such as signal latency, connectivity, and wire length, which gives us three regimes of scale. The regime that considers total wire length but neglects the speed of light is probably the least realistic regime that we consider, and we suspect that the number of connections between processors/memory is more relevant than the total length of wires. Nevertheless, we include this regime as it is the only one we found which led to potential quantum advantage.

To start the comparison, we give a lower-bound on the number of QRAM accesses that any quantum algorithm will need to solve Problem 1, then show a classical algorithm that solves the problem with the same number of matrix-vector multiplications. We can then compare parallel classical matrix-vector multiplication to a single QRAM access.

Theorem 4.1 (Quantum Polynomial Eigenvalue Transform). *Given a degree k polynomial on the interval $x \in [-1, 1]$, with $f(x) = \sum_{j=0}^k a_j x^j$, a d -sparse Hermitian matrix $H \in \mathbb{C}^{N \times N}$ and an ℓ_2 -normalized vector $|\psi\rangle \in \mathbb{C}^N$, no general quantum algorithm can prepare the state $f(H)|\psi\rangle / \|f(H)|\psi\rangle\|_2$ with asymptotically fewer than $\Omega(k)$ QRAM queries to H , each query with cost T .*

This follows immediately from Theorem A.1. The result of [49] almost immediately implies this bound, but the following derivation is included to enhance the clarity of our discussion.

The main idea is that we can efficiently construct a block-encoding (a method to encode non-unitary matrices as the top-left block of a unitary) of a rank one projector from a uniform superposition to the marked state of an unstructured search problem (specified via an oracle) with a singular value equal to $\frac{1}{\sqrt{N}}$ (where N is the dimension of the space). We can do this with only $O(1)$ queries to the unstructured search oracle; however, immediately applying the block encoding to an initial state has a low probability of success. We thus embed this in a rank-2 Hermitian matrix and apply a polynomial approximation to the sign function on the eigenvalues of this Hermitian matrix. The degree of this polynomial is $\tilde{O}(\sqrt{N})$, and it gives a constant probability of success in measuring a solution to the unstructured search problem. Consequently, if we could implement this polynomial with $o(\sqrt{N})$ queries, we could solve the unstructured search with less than $\Omega(\sqrt{N})$ complexity. Since unstructured search has an $\Omega(\sqrt{N})$ lower bound, this creates a lower bound of $\Omega(k)$ for any general quantum algorithm implementing an eigenvalue transform of a degree- k polynomial.

Theorem 4.2 (Classical Parallel Polynomial Eigenvalue Transform). *Given a degree k polynomial on the interval $x \in [-1, 1]$, with $f(x) = \sum_{j=0}^k a_j x^j$, a d -sparse Hermitian matrix $H \in \mathbb{C}^{N \times N}$, and an ℓ_2 -normalized vector $\mathbf{v} \in \mathbb{C}^N$, a parallel classical algorithm can exactly compute the vector $f(H)\mathbf{v} / \|f(H)\mathbf{v}\|_2$ with $O(k)$ matrix-vector multiplications and k vector-vector additions.*

Scale	Assumptions		Quantum advantage	
	Free wires	Instant communication	Sparse Matrices	Dense Matrices
Small	Yes	Yes	None	None
Medium	No	Yes	$\tilde{O}((Nd)^{1/2})$	None
Large	No	No	None	None

Table 1: Comparing classical versus quantum algorithms for Problem 1 for $N \times N$ matrices, under different architectural assumptions with active QRAM. It may appear strange that the potential quantum speedup in the Medium-Scale regime increases as d increases, and then suddenly vanishes when $d \in \Omega(N)$. We suspect that a classical algorithm for the medium-scale regime for sparse matrices with $d \gg 1$ could be improved.

Proof. By definition,

$$f(H) = \sum_{j=0}^{N-1} \sum_{l=0}^k a_l \lambda_j^l |\lambda_j\rangle \langle \lambda_j|. \quad (7)$$

Applying the completeness of the eigenvectors of H , we immediately get $f(H) = \sum_{j=0}^k a_j H^j$. This means we must simply compute $f(H)\mathbf{v} = \sum_{j=0}^k a_j H^j \mathbf{v}$. This can be done with k matrix-vector multiplications by computing the powers $H^1 \mathbf{v}, H^2 \mathbf{v}, \dots, H^k \mathbf{v}$, and adding $a_j H^j \mathbf{v}$ to a running total vector after each multiplication. We do not need to store all the powers $H^j \mathbf{v}$, since we can delete each one once we add it to the total and compute the next power of H . Once the unnormalised vector $f(H)\mathbf{v}$ has been constructed, we can normalize it by computing the ℓ_2 norm (which requires $O(N)$ parallelizable operations, and $O(\log N)$ joining steps), and then by dividing each element in the output vector by this norm, which requires a further $O(N/P)$ steps. We assume that the cost of matrix-vector multiplications dominates the cost of this normalization. $\square$

From now on we ignore the cost of vector-vector additions (which parallelize perfectly), since they are likely negligible compared to matrix-vector multiplications. Thus, our comparison comes down to the expense of matrix-vector multiplication. In our computational model, the QRAM with Nd registers implies a proportional number of classical processors, so we now consider highly parallel matrix-vector multiplication at different scales. Table 1 summarizes our conclusions.

Small-Scale Regime: At this scale we ignore all latency and connectivity concerns. The QRAM access can then be done in $\Theta(\log(Nd))$

time (i.e., circuit depth). We assume the classical processors can access a shared memory of size Nd , also with $O(\log(Nd))$ access time.

A standard parallel matrix multiplication algorithm (see Theorem A.2) allows the the $P = \Theta(dN)$ processors to multiply the matrix and vector in $\tilde{O}(1)$ time, matching the assumed time of the QRAM access. Together with Theorem 4.1 and Theorem 4.2, the classical algorithms achieves $\tilde{O}(k)$ scaling, while the quantum algorithm achieves a $\tilde{\Omega}(k)$ lower-bound for the general linear algebra task, so *there can be no asymptotic quantum advantage* at this scale.

Medium-Scale Regime: With a large matrix, shared memory access is unrealistic. One problem is that it requires $\Omega((Nd)^2)$ wires to connect each processor to each memory element. The QRAM does not need so many wires; a bucket-brigade QRAM needs only $O(Nd)$ wires, for example.

Thus, in this regime we only allow each processor to have a small number of wires (and count the total length of wires). Connecting the processors to each other and restricting access to local memory requires different algorithms. Theorem A.3 shows how to multiply a sparse matrix and vector using $O(1)$ sorts. With $O(\log(Nd))$ connections per processor (and thus $\tilde{O}(Nd)$ wires), the processors can form a hypercube and sort in polylogarithmic time. In this case, the time for QRAM access and classical matrix-vector multiplication match, again negating any quantum advantage.

However, we should also consider the *length* of the wires. By “wire” we mean any communication infrastructure, under the assumptions that (a) two connections cost twice as much as one connection, and (b) a connection which is twice as long will cost twice as much. This means the

architecture needs more resources to have longer connections. A real architecture must embed into the 3-dimensional Euclidean space we live in, and we further assume it embeds into 2-dimensional space to account for heat dissipation. Each classical processor has a finite size, so the total width of this machine must be $\Omega((Nd)^{1/2})$. This means the length of wires in this hypercube scales to $\tilde{\Omega}((Nd)^{3/2})$, which is also infeasible.

The total width of the QRAM must also be $\Omega((Nd)^{1/2})$ by the same reasoning, but the QRAM need only have $O(Nd)$ total wire length if it is well-designed (e.g., an H-tree).

To use $O(Nd)$ wires classically, we will assume the classical processors are in a two-dimensional mesh with nearest-neighbour connectivity. In this case a sort takes time $O((Nd)^{1/2})$, so the time for matrix-vector multiplication is $\tilde{O}((Nd)^{1/2})$, compared to $\tilde{O}(1)$ for QRAM access. Thus, we have a square-root time advantage for the QRAM in this case.

This only applies to sparse matrices, however. If the matrix is dense, the QRAM implies $\Omega(N^2)$ classical co-processors, which can multiply the matrix by a vector in time $O(\log(N))$, as we show in Theorem A.4. This means the quantum advantage can only potentially apply to sparse matrices.

Strangely the quantum advantage increases with d until vanishing when $d = \Omega(N)$. Likely this is because the classical hardware budget increases with the QRAM size, which increases with d , and we have not considered optimized classical algorithms for large d . The sparse case is harder for the classical machine because the memory access for small d could be random, creating a large routing problem to send data from arbitrary memory cells to each processor.

Large-Scale Regime: At large enough scales, we must account for the time for signals to propagate through the computers, even at lightspeed. This does not change the asymptotic time for the 2-dimensional mesh, which only has short local connections. The QRAM must have a width of $\Omega((Nd)^{1/2})$, so each QRAM access now takes time $\Omega((Nd)^{1/2})$. Thus, the classical and quantum algorithms for Problem 1 both take time $\tilde{\Theta}(k(Nd)^{1/2})$, and the asymptotic quantum advantage disappears.

Which regime is realistic? The scale of QRAM

necessary for these algorithms would depend on the application, but for context, Google’s “IMAGEN”, a text-to-image diffusion model, used 2^{43} bits of data for its training set [99].

The small-scale regime might hold up to $Nd \approx 2^{29}$, as [14] ignore memory connectivity and compute sparse matrix-vector multiplication at that size in 0.03 s with a single CPU and GPU. Larger problems will certainly face bandwidth issues, moving us to at least the medium-scale regime. The large-scale regime might seem far-fetched, where lightspeed signal propagation is the limiting factor, but consider that light would take 173 clock cycles to travel across the Frontier supercomputer, which has “only” 2^{56} bits of memory [38]. Frontier also has enough wire that any two nodes are at most 3 hops away [38].

Further, we might be somewhere in between. We can divide a matrix into a block matrix, and use small-scale techniques for the blocks themselves and a larger-scale algorithm for the full matrix.

In practice, Google’s TPUs are 2-dimensional grids of processors with only nearest-neighbour connectivity, optimized for dense matrix multiplication [54]. Instead of the techniques we describe, they use a “systolic” method to multiply an $N \times N$ matrix that takes time proportional to N with N^2 processors. Their architecture is optimized for *throughput*, as it can simultaneously multiply the same matrix by many vectors. The throughput asymptotically approaches $O(1)$ cycles per matrix-vector multiplication, the best one can do with N^2 processors.

Overall, this becomes a battle of constant factors. The quantum advantage for sparse matrix linear algebra in the medium-scale regime depends on the constant factor overheads of quantum computing being less important than the constant factor of signal latency.

4.4 Quantum Linear Algebra with Noisy QRAM

The opportunity cost arguments made in the last section apply for any active QRAM system, but they are strongest for error-corrected architectures (which require significant classical co-processing). In contrast, passive QRAM systems (potentially such as certain implementations of bucket-brigade) suffer no such opportunity cost, as once the QRAM query is initiated, the sys-

tem requires no further energy input or intervention (although they will necessarily be noisy). As such, if sufficient progress can be made in the construction of a passive QRAM system (despite the challenges we identify in Section 6), then for certain linear algebra algorithms that can tolerate some noise in their QRAM queries, it is possible that practically relevant quantum advantage could be realized.

For example, with the bucket brigade architecture, the overall probability of an error per query to a QRAM with N memory cells scales as $\sim p \log(N)^2$, where p is the physical probability of an error (see Section 9.4,[59]). This immediately rules out an asymptotic advantage for any quantum algorithm asymptotically improving error-rate dependence (for arbitrarily small errors), but may be practically useful in cases where either the overall algorithm is relatively insensitive to QRAM access errors, or if the physical error rate can be made to approach the necessary precision without error correction or external intervention.

In some linear algebra tasks, a small constant error per QRAM access *may* be fine. For instance, in some machine learning applications the underlying data is already noisy, so additional noise may be acceptable. In other cases, such as the training and deployment of large language models (LLMs), it has been demonstrated that models can still function with reasonable accuracy when the weights are quantized down to 16-bits [67], 8-bits [41], or *even down to 2-bits* [44]. Clearly a two-bit quantization introduces substantial noise in the evaluation of a model, and if LLMs can still function in this regime, it is possible they could be similarly resilient to noise in a QRAM query to their parameters.

Moreover, in reinforcement learning, additional noise could assist an agent to explore its environment, rather than just exploit its existing policy [105].

For matrix inversion with a structured matrix (i.e. a matrix not stored in QRAM) on an input vector constructed from QRAM, for well-conditioned matrices the output of the matrix inversion is relatively insensitive to small perturbations in the input vector. For matrices in QRAM, for some classes of matrices the inversion could be insensitive to perturbations in the structure of the matrix itself [42].

We recommend further exploration into error-corrected quantum linear algebra algorithms using noisy QRAM systems. We do not rule out the possibility of practical quantum advantage in such cases. Of course, at some scale the uncorrected noise will overwhelm nearly any algorithm, but with sufficient noise resilience (in both e.g. the bucket-brigade QRAM and in the algorithm itself), at realistic scales such quantum algorithms could still be of significant commercial and scientific value.

5 Circuit QRAM

Here we highlight four approaches to circuit QRAM and give brief descriptions of each. For the full details, follow the references. All of these approaches have overall costs of $\tilde{\Theta}(N)$ gates to access N bits of memory, even with logarithmic circuit depth. This may feel unreasonably high; however, in a bounded fan-in circuit model, this is essentially optimal.

As Section 3 shows, there are many applications where QRAM is a useful tool, even with this high cost. Circuit QRAM is a valuable tool in quantum algorithm analysis.

5.1 Lower Bounds

The key point of these lower bounds is that with N bits of memory in a QRACM, there are 2^N possible QRACM gates, depending on the values in those bits. However, the number of different quantum circuits is roughly exponential in the product of the number of qubits and the circuit depth, which one can see because we have a finite set of choices of gate for each qubit at each time step. Thus, we need $N \approx \text{depth} \times \text{width}$. This is analogous to a classic result of Shannon about classical circuits [102].

More precisely, we prove the following Lemma in the appendix:

Lemma 5.1. *The set of all distinct quantum circuits on W qubits of depth D using G gates from a fixed set of g possible gates of fanin at most k , has size at most $2^{\gamma G}$, where $\gamma = k \cdot \lg(\frac{DWg}{G\sqrt{k}})$.*

Intuitively, one can obtain a simple non-rigorous version of the above result with the following argument. Suppose we have a circuit with G gates, and that the gate set contains $M = 2^m$

possibilities, including the identity. Then, there are $M^G = 2^{mG}$ possible circuits. The above argument makes this notion rigorous and general, e.g., covering cases where some gates may be multi-qubit controlled gates.

We will prove the lower bound even for approximations of QRACM, which we define as follows: Given a table T with N rows, let $U_{QRACM}(T)$ be the QRACM circuit implementing T . A circuit C ϵ -approximately implements a $U_{QRACM}(T)$ if $\|C - U_{QRACM}(T)\|_2 \leq \epsilon$.

Theorem 5.2. *Let $0 \leq \epsilon \leq 1/\sqrt{2}$. Let $0 < \tau \leq 1$ be the fraction of possible QRACM tables that circuits with at most G gates from the gate set in Theorem 5.1 is able to ϵ -approximately implement. Moreover, define γ as per Theorem 5.1. Then,*

1. *A given circuit C can ϵ -approximate at most 1 QRACM table.*
2. *The number of gates G to ϵ -approximately implement a fraction τ of all possible QRACM tables is lower-bounded by $G \geq \frac{\log \tau + N}{\gamma}$. That is, $G \in \tilde{\Omega}(N + \log \tau)$.*
3. *To approximately implement all possible QRACM tables with circuits using at most G gates, $N \in \tilde{O}(G)$, and equivalently, $G \in \tilde{\Omega}(N)$.*

Proof. We are given two N -row QRAM tables T and T' , enacted by the unitaries $U(T)$ and $U(T')$ respectively. Assume that the tables differ at index j , i.e., $T[j] \neq T'[j]$. Then,

$$\|(U(T) - U(T'))|j\rangle_n |0\rangle_1\| = \sqrt{2}. \quad (8)$$

Consequently, $\|U(T) - U(T')\|_2 \geq \sqrt{2}$.

Now suppose that a given unitary circuit C ϵ -approximates both $U(T)$ and $U(T')$, i.e., $\|U(T) - C\|_2 \leq \epsilon$ and $\|U(T') - C\|_2 \leq \epsilon$. Then,

$$\sqrt{2} \leq \|U(T) - C + C - U(T')\|_2 \leq 2\epsilon. \quad (9)$$

Thus, $1/\sqrt{2} \leq \epsilon$. Thus, it is not possible for one circuit to approximate more than one QRAM table, unless $\epsilon > 1/\sqrt{2}$. Importantly, **allowing tables to be approximated to a constant error does not provide any meaningful asymptotic advantage.**

Using the gate set defined in Theorem 5.1, Theorem 5.1 shows that there are $2^{kG(\lg(\frac{DWg}{G\sqrt{k}}))}$

possible circuits using G gates. Assuming that $\epsilon \leq 1/\sqrt{2}$, each distinct circuit can only approximate at most a single QRAM table. Let $\gamma := k(\lg(\frac{DWg}{G\sqrt{k}}))$. Consequently, a set of $2^{\gamma G}$ distinct circuits can approximate at most $2^{\gamma G}$ distinct QRAM tables.

Noting that there are 2^N possible QRAM tables with N rows, the fraction of ϵ -approximable tables with at most G gates is bounded by $\frac{2^{\gamma G}}{2^N} = 2^{\gamma G - N}$. Let $0 < \tau \leq 1$ represent the fraction of QRAM tables that circuits with G gates are able to approximate. Then,

$$\tau \leq 2^{\gamma G - N} \implies \frac{\log \tau + N}{\gamma} \leq G. \quad (10)$$

Thus, for circuits with G gates to be able to implement a τ fraction of all possible N row QRAMs, $G \in \tilde{\Omega}(N + \log \tau)$. $\square$

We will now briefly outline some special cases of this result, to help build intuition. If we wish to:

- Cover all possible tables, then $\tau = 1$, so $G \in \tilde{\Omega}(N)$.
- Cover a constant fraction of possible tables then $\tau \in \Theta(1)$, so $G \in \tilde{\Omega}(N)$. E.g., if we wish to cover exactly half of all possible QRAM tables, $\tau = 1/2$, so $\frac{N-1}{\gamma} \leq G$. If we wish to cover a quarter of all possible tables, $\frac{N-2}{\gamma} \leq G$, etc.
- Cover an exponentially small fraction of tables, e.g., $\tau \in \Theta(2^{-N})$, so $G \in \tilde{\Omega}(1)$. Let $c > 0$ be some constant such that $\tau = c2^{-N}$. Then, we find $\log_2(c)/\gamma \leq G$.

This demonstrates how this result generally applies, even in cases where QRACM circuits can be efficiently constructed, as such efficient QRACM circuits belong to the exponentially small fraction of circuits where the gate-complexity lower-bound is efficient.

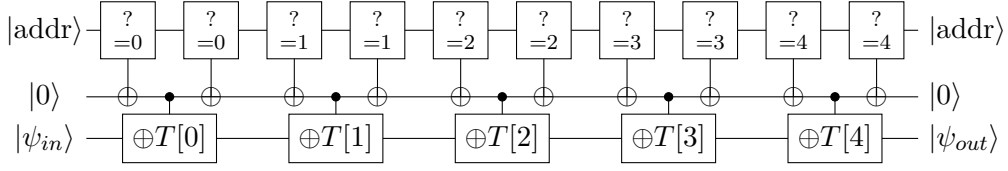


Figure 2: Circuit diagram for naive QRACM. Here $\stackrel{?}{=} j$ means a circuit that checks if the input equals j , and flips the target output if so.

Algorithm 1 Simple implementation of a naive QRACM access.

- 1: **Input:** A qubit address register, an output qubit, and a classical table T of N bits
- 2: **for** $i = 0$ to N **do**
- 3: **if** $T_i = 0$ **then**
- 4: Do nothing
- 5: **else**
- 6: Apply a circuit to check if the address register equals i and write the result to an ancilla $|b\rangle$
- 7: Apply a CNOT from $|b\rangle$ to the output qubit
- 8: Uncompute $|b\rangle$
- 9: **end if**
- 10: **end for**

Notably, the lower bound applies to *circuits*, meaning measurements are forbidden. With feedback from measurements, our simple counting arguments fall apart quickly: consider the set of operations, each defined by a table of size N , where one measures the entire input, classically finds that address in the table, then applies an X if the bit at that address is 1. It’s easy to see that there are 2^N distinct “operations” of this type, but the number of quantum gates to implement each one is only $\lg(N) + 1$. However, these are not QRACM. To be useful, the measurements need to be mostly independent of the input state, but we leave it to future work to incorporate this into the counting arguments. If the feedback from measurements is simple enough to replace it with quantum control without drastically increasing the resource requirements, the bounds will hold. [112] provide a similar lower bound through different techniques which are also restricted to measurement-free circuits.

This lower bound implies that variational approaches to QRAM ([87, 93]) cannot scale better than other circuits. We can discretize the continuous parameters to get a finite gate set, which increases the parameter g in Theorem 5.1, but does

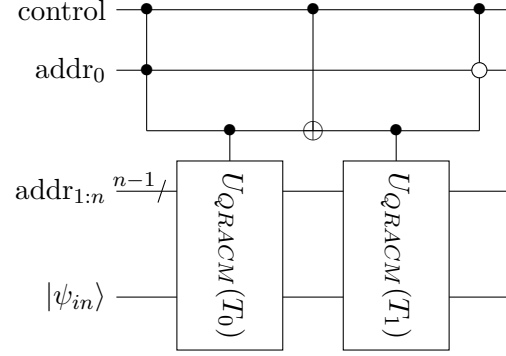


Figure 3: Recursive controlled unary QRACM from [8].

not appreciably change the asymptotics until the precision of the parameters becomes exponentiation in N , the table size. Such precision seems absurd: if the duration of a laser pulse were specified to such a high precision, it would take $\approx N$ bits just to transmit the duration of the pulse to the laser!

5.2 Unary Encoding

In a QRACM access, the memory is entirely classical, so the control program can access all of it. The control program can dynamically construct a circuit to represent a particular memory. A naive strategy is to “check-and-write” for each address, as in Algorithm 1/Figure 2:

The cost is $O(N \lg N)$ gates and the depth can be as low as $O(N \lg \lg N)$ if we use auxiliary qubits for the address comparison. This is already nearly optimal.

[90] present approximately this version, while the optimized version of [8] use an efficient recursive structure of *controlled* QRACM. The base case, a one-element controlled look-up, is just a CNOT. For the recursive case, see Figure 3, where T_b is a table of size $N/2$ defined from a table T by taking all elements whose address starts with the bit $b \in \{0, 1\}$. This uses $\lg N$ ancilla qubits and a low-T AND gate to give a T-count of $4N - 8$, with a total gate cost of $O(N)$.

Attractive features of this circuit are its low qubit count and its simplicity. Even better, if the table has multi-bit words, then the base case – a controlled look-up in a one-element table – is just a multi-target CNOT gate. This has constant depth with only Clifford gates and measurements, so the overall costs for an m -bit table are $O(Nm)$ gates in depth $O(N)$.

5.3 Bucket-Brigade

Bucket-brigade QRAM [52] was a groundbreaking approach to QRAM that laid the basis for almost all hardware-QRAM since. It is not often considered in the circuit model (though see [59, 81, 89]).

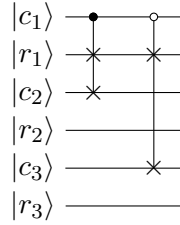
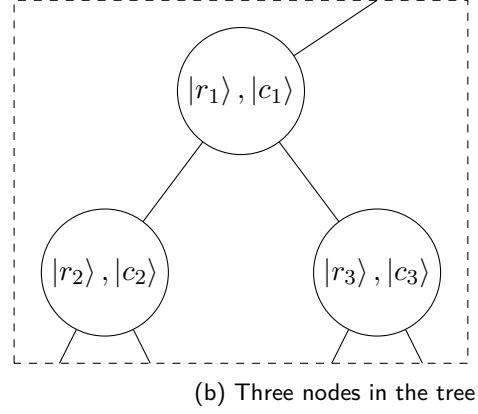
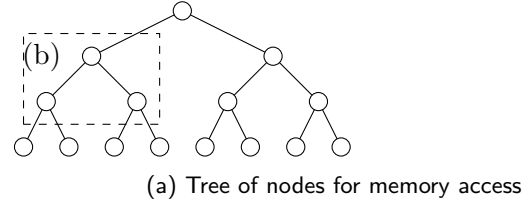
The core idea is a binary tree, and we imagine “sending in” address bits. At each node in the tree, the first bit to reach that node sets the state of the node to either “left” or “right”, depending on whether the bit was 0 or 1. Any subsequent bit reaching that node is directed either left or right, accordingly.

In the circuit model, we copy the description of [59], shown in Figure 4. Each node of the routing tree has two qubits: a routing qubit $|r_i\rangle$ and a control qubit $|c_i\rangle$, all of which are initialized to $|0\rangle$.

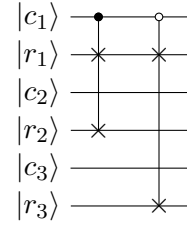
To perform a memory access, we start by swapping the top (most significant) bit of the address into the control qubit at the root node of the tree. We then swap the second-highest address bit into the *routing* qubit at the root node, then apply the routing circuit shown in Figure 4d. This swaps the routing qubit (which now contains the state of the second-highest address qubit) into the control qubit of either the left or right node in the next level, depending on the state of the control qubit of the root node.

We then repeat this process: we swap the i th address bit into the routing qubit at the root, then apply the routing circuit to all nodes at the 0th level, then the 1st level, up level $i - 1$. Then, at the i th level, we apply the control circuit in Figure 4c. This puts the address qubit into the control, so it can properly route the next address qubit.

At this point, there are many possible approaches to read the actual data. For QRAM, one could use X gates to the i th leaf of the table if $T[i] = 1$. We then apply the routing circuit to all nodes in each layer, but iterating from the



(c) Control circuit.



(d) Routing circuit.

Figure 4: Schematic of the overall tree structure of circuit bucket-brigade QRACM. In the tree structure in (a), the lines between nodes are only to help exposition and do not represent any physical device (unlike in hardware bucket-brigade QRACM; see Section 9). (b) shows how each node consists of 2 qubits, and labels them to show where the circuits in (c) and (d) apply their gates.

bottom layer to the top. This will swap the state of the qubit at the desired address back to the root of the tree, where we can copy it out with a final CNOT. Then the entire circuit must be uncomputed.

To create QRAQM, if we treat the table of memory as the bottom layer of the tree, the same SWAP technique as for QRACM will enact a swap QRAQM.

Cost. Both the control and routing circuits require 2 controlled-SWAPs at each node. For the i th address bit, we must apply one of these circuits to *every* node from the root down to the i th layer. The layered structure of gate applications

gives a total count of $2(n \cdot 2^0 + (n-1)2^1 + (n-2)2^2 + \dots + 2^n) = O(N \log N)$, where $N = 2^n$.

Naively the depth is $O(\log^2 N)$, since each address bit requires looping over all previous layers. However, as soon the i th address bit has been routed into the 2nd level of the tree, address bit $i+1$ can be swapped into the root and start that layer. Thus, the depth is $O(\log N)$.

This circuit requires $2(N-1)$ ancilla qubits just for the routing tree, and may require more for the data itself, depending on how the final read is designed.

Overall, the costs are log-linear, but worse than unary QRACM in constant factors. While the depth is only logarithmic in N (the best known), there are many approaches to a log-depth circuit QRAM [32, 34, 65].

Error scaling. An advantage of the bucket-brigade approach is the favourable error scaling, proven in [59] and summarized in Section 9.4. Each node only needs errors below $O(1/\text{polylog}(N))$, rather than $O(1/N)$.

In a surface code architecture, this favourable error scaling means we could reduce the code distance for the qubits in the bucket-brigade QRACM. This might noticeably decrease overall depth, physical qubits, and perhaps allow easier magic state distillation. Of course, it would need to exceed the losses from the less efficient circuit design compared to (e.g.) a unary QRACM. Further, it would only benefit applications where the depth of the QRACM access is more important than the qubit count, since the bucket-brigade and its error scaling depend on a fast, wide circuit. Depth-limited cryptographic attacks are an appealing candidate application. Quantifying where this advantage lies is a fascinating question for future work.

5.4 Select-Swap

[79] describe a technique that reduces T-count even further, at the expense of more ancilla qubits (which can be in an arbitrary state, however). The technique appears also in [40] and [19]. This method is analogous to “paging” in classical memory. In most classical memory devices, it is just as efficient to retrieve larger chunks of memory (called “pages”) as it is to retrieve single bits. Hence, the memory device retrieves a page and a second device will extract

the specific word within that page. Analogously, this QRACM circuit uses a QRACM circuit on a larger page of the classical table, then uses a QRAQM access on that page.

Recall that the T-gate cost of unary QRACM (and depth) is independent of the length of each word in the memory table. From any table T with N entries in $\{0, 1\}$, we can construct a table T' with $N/2^\ell$ entries in $\{0, 1\}^\ell$, simply by grouping ℓ consecutive entries – a page – as one word in T' . Thus, to begin access to T , we start with a unary QRACM access to T' , writing the output in an 2^ℓ -bit auxiliary register. We do not need the last ℓ bits of the address to do this, since the page we retrieve has every element that the last ℓ bits could address.

Once this is done, we want to extract the precise bit that the last ℓ bits address. Since the required page is already in the auxiliary register, we use a QRAQM access to do this. The address in the page for this QRAQM access is the last ℓ bits of the original address, and we write the output to the original output register.

After this, we uncompute the QRAQM access and uncompute the original unary QRACM access.

To see why this works, notice that after the first QRACM access, the 2^ℓ auxiliary bits are now in a superposition, with the contents of these bits entangled with the high bits of the address register. Thus, a *single* QRAQM access extracts the correct output for each of these states in superposition.

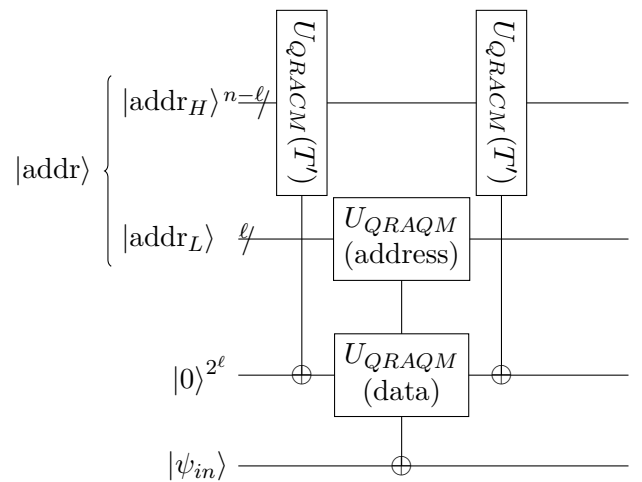


Figure 5: Select-swap QRACM access. T' is the table formed from T by combining each page of 2^ℓ consecutive bits of T into one word in T' .

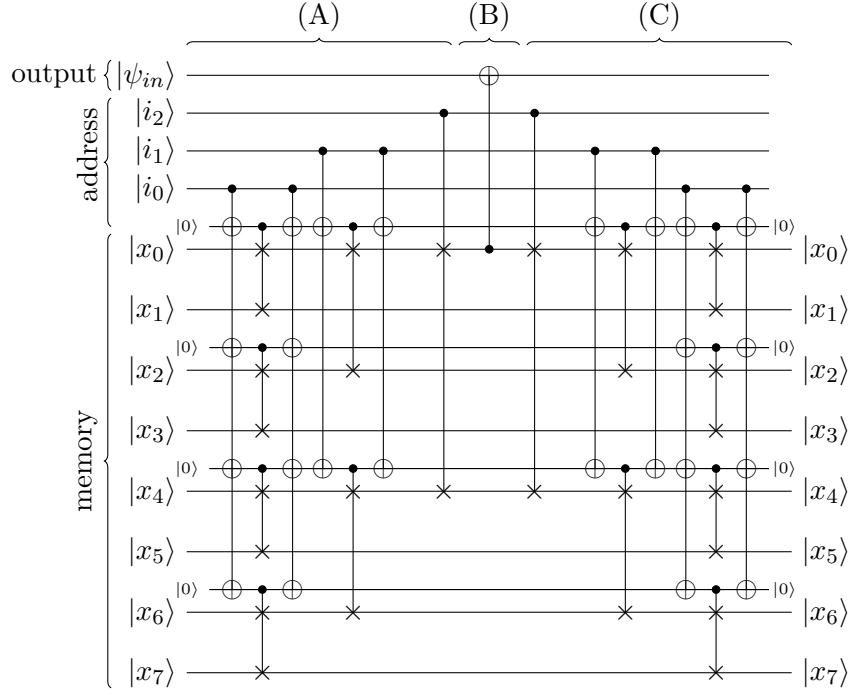


Figure 6: Fanout-and-swap QRAQM for $N = 8$. (A) uses a binary tree of swaps to move $|x_i\rangle$ into the register for $|x_0\rangle$, where (B) copies it out (replacing this CNOT with a SWAP would produce a SWAP-QRAQM). Then (C) restores the memory register to its original state.

Cost. The cost of the first unary QRACM access is $O(N/2^\ell)$ T-gates, in depth $O(N/2^\ell)$ generally. For the second QRAQM access, it needs $\tilde{O}(2^\ell)$ gates and has depth $O(\ell)$. To minimize T-gates, a page size of $2^\ell \approx \sqrt{N}$ gives a total T-cost and depth of $O(\sqrt{N})$.

However, notice that the total gate cost is still proportional to N , since writing each word of T' requires 2^ℓ CNOT gates. Thus, the majority of the gates in this circuit are used for multi-fanout CNOTs. If these are cheap, then this circuit is cheap overall.

The qubit requirements for the T-gate-optimal parameters are also proportional to $\sqrt{N}$. Using other parameters, this approach provides a nice extrapolation between bucket-brigade and unary QRACM, able to reach almost any combination of depth and width whose product is $\Theta(N)$. Even better, this method can use dirty qubits (i.e., qubits which may hold data necessary for other parts of the algorithm) for all but one word in each page.

The circuit in the appendix of [19] achieves this asymptotic cost, but they use a measurement-based uncomputation in both the QRAQM and QRACM access. This means they measure each ancilla qubit in the $\{|+\rangle, |-\rangle\}$ basis, giving a bi-

nary string $\mathbf{v}$. If the ancilla qubits were in state $|b\rangle$ for $b \in \{0,1\}^m$, then that state acquires a phase of -1 if and only if $\mathbf{v} \cdot \mathbf{b}$ has odd parity (and extend this linearly). They clean up this phase with a second look-up. Since this phase is a single bit, this is cheaper than a forward computation when the words of T are larger than one bit. For single-bit words they also reduce uncomputation costs by $\frac{1}{2}$.

This optimizes the non-asymptotic gate counts. They use a QRAQM circuit based on controlled swaps, rather than the bucket-brigade; see Figure 6 for an analogous technique that keeps depth and T-count small by using high-fanout CNOT gates. The circuits in [19] achieve the lowest T-count of any QRACM circuit to date.

[65] provide a T-depth optimized method to convert binary to unary encodings which could likely be combined with [19]. [40] decompose the SWAP gates to apply T-gates in parallel to achieve a similarly low T-depth.

5.5 Parallel QRAM

Taking a step back to the algorithm calling QRAM, many algorithms call the gate repeat-

edly and thus could benefit from parallelization. We define such a gate as follows:

Definition 3. A parallel quantum random access classical memory is a set of unitaries $U_{PQRACM}(T)$, parameterized by a table $T \in \{0,1\}^N$, such that for any k address registers $|i_1\rangle \dots |i_k\rangle$, with $1 \leq i_j \leq N$ for all $1 \leq j \leq k$,

$$U_{PQRACM} |i_1\rangle |0\rangle |i_2\rangle |0\rangle \dots |i_k\rangle |0\rangle \\ = |i_1\rangle |T_{i_1}\rangle |i_2\rangle |T_{i_2}\rangle \dots |i_k\rangle |T_{i_k}\rangle. \quad (11)$$

We can define parallel QRAQM similarly, with a shared external register $|T_0\rangle \dots |T_{N-1}\rangle$.

Clearly one can enact this by independently applying QRAM to each address register, for a total gate cost of $O(kN)$. However, since the calls share the same table, the cost can be reduced. [13] solve this with sorting networks.

To briefly explain: they augment the table T to a table T' by adding the address of each element to that element, so each entry is $(i, T_i, 1)$. Then they augment the address registers to $(i_j, 0, 0)$ and write this entire table into quantum memory. After this, they sort all of the memory registers *and* the address registers together, using an ordering based on the first address register, breaking ties with the final flag register (which is 1 for memory elements and 0 for query elements).

Sorting with this ordering ensures that the quantum memory is laid out as

$$\dots |i, T_i, 1\rangle |i+1, T_{i+1}, 1\rangle |i+2, T_{i+2}, 1\rangle |i+2, 0, 0\rangle \dots \quad (12)$$

where one can see that a query with address $i+2$ has been placed adjacent to the memory element at address $i+2$, which the query needs to access. From there, a careful series of controlled CNOTs copies each memory element into any adjacent queries. This takes some care: at this point the control program does not know whether a given register of qubits contains a memory element or a query, so it must use a control made from the last flag bit of both words as well as whether they have the same address. Moreover, multiple queries may access the same memory element, so the copying must carefully cascade memory elements forwards whenever adjacent registers have the same adders.

Once this is all done, they unsort the network. Now the address registers have their requested data. A quantum sort must keep a set of ancilla qubits recording the outcome of the comparisons

of the sort, for reversibility, so these are used and uncomputed to reverse the sort. In some sense, the newly copied data just “follows” the rest of the sort.

The cost of this scheme is primarily the cost of the sorting network, which in turn is governed by the connectivity of the quantum computer architecture. [13] emphasize that with a connectivity that they call the “hypercube” (defined to mean that if we address each register by a bitstring, each is connected to registers whose address differs in only one bit), sorting N memory elements and k queries requires only $O((N+k)\log(N+k))$ gates and depth $O(\text{polylog}(N+k))$. However, with a two-dimensional nearest-neighbour architecture (e.g., a surface code with no flying qubits), it requires $O((N+k)^{3/2})$ gates and $O((N+k)^{1/2})$ depth. It remains debatable whether hypercube connectivity is feasible; we discuss this some in Section 8.4.

5.6 Data Structures

Classical computers use RAM to store and access data in larger data structures such as linked lists, hash tables, etc. Analogous quantum data structures would inherently use QRAQM, such as the quantum radix trees in [18]. However, with a QRAQM access cost proportional to N in the circuit model, bespoke data structure circuits likely exist.

As an example, [63, Section 4] performs insertion into a sorted list by making N copies of an input with a fanout gate, then simultaneously comparing *all* elements of the list to a copy of the input, then shifting all elements of the array. This requires N comparisons, but no explicit QRAQM access. [46] takes a similar approach to build a quantum dictionary.

We encourage algorithms requiring quantum data structures to use similar custom circuits to operate on the data, rather than building them from QRAQM accesses.

6 Hardware QRAM

Here we consider a specialized device for QRAM, in the model of *hardware-QRAM*, that is not built from general-purpose qubits. The gate lower bounds for circuit QRAM are unpalatable for certain applications, but there is no reason that all

parts of a future quantum computer are built as circuits in a small gate set. Like GPUs or RAM in classical computers today, it's easy to imagine specialized devices for specific tasks. The goal of hardware-QRAM is something much more efficient than implementing QRAM access as a circuit.

This section asks whether such a specialized device can be built and run efficiently. Not all tasks can have specialized hardware; it's unreasonable to expect a sub-module which solves 3-SAT in quantum polynomial time. Our question is whether QRAM access is a simpler problem.

To give a quick sense that QRACM is generally difficult, consider that if the table is $T = [0, 0, 0, 1]$, then QRACM access to this table is equivalent to a Toffoli gate. Hence, Clifford+QRAM is a universal gate set.

In this section we lay out several issues that any proposed QRAM device must address to be competitive with circuit-based QRAM.

6.1 Active vs. Passive QRAM

Recall that in the memory peripheral model, we cost quantum operations by the number of interventions by the classical controller. Here we extend that slightly to include the energy cost of the external controller. For example, we would count the cost to run a laser.

An *active* QRAM gate is one with cost in this sense that is at least proportional to N for each QRAM access. Active hardware-QRAM will still fail asymptotically in the same way as active circuit QRAM. Recalling the scale of memory in Section 3, even if the constant in this asymptotic term is tiny, it quickly adds up when the memory sizes are 2^{50} bits or larger.

There may be a constant-factor advantage to active hardware-QRAM, but Section 5 shows that the constants for circuit QRAM are already low. Moreover, when comparing a quantum algorithm using QRAM to a classical algorithm (such as with machine learning), the quantum algorithm must overcome a great deal of overhead to have advantage (e.g., Google's quantum team estimates that a quadratic advantage is insufficient in the near term [9]; error correction nearly eliminates the advantage of quantum lattice sieving [3]). If QRAM has even a small linear cost, algorithms like database HHL and database QAA will only have – at best – a constant-factor

advantage over classical.

For these reasons, we discount any active hardware-QRAM, though we cannot rule out a constant-factor advantage from such a device.

Notice that passive QRAM (with $o(N)$ cost for each access) needs both the readout and routing stages to be passive. Even if our readout is quite low-energy – say, if we can read directly from a non-volatile classical memory – if the routing stage has cost $\Omega(N)$, the entire QRAM access would have cost $\Omega(N)$.

6.2 Path Independence

Accessing memory in superposition means that we must not destroy any of the states in superposition, which means that no external system, including the controller enacting the memory, can learn *anything* about the memory access itself. A major obstacle here is the path in which the memory access travels.

In contrast, consider one aspect of classical memory access that QRAM cannot emulate. If there is a tree of routing nodes as in the bucket-brigade QRAM, a classical computer can keep all of the nodes in a low- or zero-power “idle” mode, where a node activates only when it receives a signal from its parent node. Such a device would consume energy only in proportion to $O(\log N)$, not $O(N)$. QRAM cannot do this because in general it needs to be able to act on an arbitrary superposition of addresses, and we cannot know the support of the given superposition non-destructively. If we learn that a path was taken, this collapses the state. Thus, if a QRAM gate requires any active intervention (such as a controlled swap like the circuit QRAM), then the controller *must* apply that intervention to *all* nodes, immediately bringing the (parallelizable) cost to $\Omega(N)$.

In their description of bucket-brigade QRAM, [52] describe an “active” path through the tree for each address. If a node requires some external intervention (such as energy) to perform its role in the device, then *all* nodes must receive that energy or intervention. That is, the “activation” of a node is all-or-nothing: if we cannot build a device where all nodes can route data passively (such as [29] attempts), then all nodes must be active during a memory access.

6.3 Hamiltonian Construction

If a QRAM gate is *completely* passive, i.e., requires no external intervention, then this implies that the Hamiltonian describing the entire QRAM device must be constant throughout the operation of the gate, since any change to the Hamiltonian requires external intervention. This means there must be some Hamiltonian $H_{QRACM}(T)$ and time t such that (let $\hbar = 1$)

$$U_{QRACM}(T) = e^{iH_{QRACM}(T)t}. \quad (13)$$

We now ask how difficult it will be to establish such a Hamiltonian. If we need to reconstruct the Hamiltonian every time we access the QRAM, then the number of terms (and hence the number of interventions) gives a lower bound on the cost.

If changing the system's Hamiltonian needs $\Omega(N)$ interventions (e.g., if we need photon pulses on all nodes of a routing tree), then our QRAM is active. We could instead try to design a system where we can evolve to the desired Hamiltonian with fewer interventions, but we will now sketch the limitations of this approach.

Firstly, we show that a Hamiltonian implementing QRACM access must be fairly complex, by recalling previous results on simulating Hamiltonians with circuits, and using the previous circuit lower bound from Theorem 5.2.

Lemma 6.1. *Let $H = \sum_{j=0}^{n-1} a_j h_j$ be a Hamiltonian where each h_j is an m -qubit Pauli string, with n such terms, acting on W qubits. Let $\mathbf{a} := (a_0 \dots a_{n-1})^T$. If e^{-iHt} enacts a QRACM unitary for a memory with N rows (i.e., with $\log_2 N$ address bits) then to be able to implement any possible QRACM table, $N \in \tilde{O}(Wn \|\mathbf{a}\|_1 t)$.*

Proof. We will first show how existing results allow for any such Hamiltonian to be simulated in the circuit model. Each Pauli string is unitary, so it is a trivial $(1, 0, 0)$ -block-encoding of itself. We can straightforwardly construct a state preparation unitary U with $U|0\rangle = \sum_{j=0}^{n-1} \sqrt{a_j} |j\rangle / \left\| \sum_{j=0}^{n-1} \sqrt{a_j} |j\rangle \right\|_2$, using $O(n)$ circuit depth. Let $b := \lceil \log_2(n) \rceil$. Then, using [48, Lemma 52] we can obtain a $(\|\mathbf{a}\|_1, b, 0)$ -block-encoding of H with $O(n)$ circuit of depth, and then [48, Theorem 58] gives us a $(1, b + 2, \epsilon)$ -block-encoding of e^{iHt} with a total circuit depth

of $O(n \log_2(n)(\|\mathbf{a}\|_1 t + \ln(1/\epsilon)))$ using $O(\log_2(n))$ auxiliary qubits.

Clearly, this block-encoding is an ϵ -approximate implementation of the QRACM gate e^{iHt} (as per Theorem 5.2). Moreover, this circuit has $\tilde{O}(Wn(\|\mathbf{a}\|_1 t + \ln(1/\epsilon)))$ total gates. Picking a fixed constant accuracy $\epsilon \leq 1/2$, we get the total gate count of $\tilde{O}(Wn \|\mathbf{a}\|_1 t)$. For a fixed precision, Theorem 5.2 gives the lower-bound of $G \in \tilde{\Omega}(N)$, and so we can conclude that circuits constructed in this way could only approximate all possible QRACM tables if the table size is at most $N \in \tilde{O}(Wn \|\mathbf{a}\|_1 t)$, meaning the Hamiltonians constructed in this way can also implement only these QRACM tables. $\square$

Theorem 6.1 shows that if our computational device pays a cost for each term it adds to the Hamiltonian, and the device must construct the Hamiltonian for each QRAM access, the cost scales just as badly as active QRAM.

One way to escape this limitation would be to construct a device where we only need to construct the Hamiltonian *once*, so the number of interventions (n) does not govern the cost to *use* the QRAM. One approach would be a Hamiltonian such that there is an “idle” eigenstate and we only need to perturb the system to start it. For example, one of the proposals for bucket-brigade QRAM imagines excitable atoms at each node, excited by input photons. Adding a photon would start the system on the desired evolution.

Feynman's original proposal for a quantum computer [43] (with Kitaev's refinement [71]) was a large Hamiltonian that evolved on its own – i.e., ballistic computation. Practical difficulties of this scheme mean it has not been experimentally explored (to the best of our knowledge). Some interesting questions that bear on QRAM include:

1. Can we add more than one 2-local term to the Hamiltonian with one intervention from the controller and energy cost $O(1)$? If not, then constructing a Hamiltonian for QRAM will cost $\Omega(N)$, as the Hamiltonian will need at least $\Omega(N)$ terms just to interact with all bits of memory.
2. What is a physically realistic error model for the Hamiltonian terms themselves? Unlike discussions about the error in the input state

to a ballistic computation, here we consider the errors in the Hamiltonian itself, i.e., how even a perfect input might stray from the desired computational trajectory. Certain proposed QRAM devices partially address this; see Section 9.4.

Overall, we highlight the notion of a single Hamiltonian for the QRAM to exemplify the extreme engineering challenges of such a device, which are far above those of even a large-scale fault-tolerant quantum computer.

7 Errors

Defining the exact error rate of QRAM is tricky, since there are several definitions of the error rate of a channel (e.g., average fidelity vs. minimum fidelity). For applications like quantum look-ups, average fidelity is likely more relevant because the look-up accesses many addresses simultaneously, whereas for database QAA, we need something like minimum fidelity.

Generally, we will use these metrics to measure the quality of the QRAM gates and their components, though somewhat imprecisely. “Error rate” will refer to the infidelity, $1 - F(\cdot, \cdot)$. For the overall QRAM, we want to compare the output of our real QRAM device (modeled as a quantum channel) to the ideal behaviour: a unitary channel defined by $U_{QRACM}(T)$ or $U_{QRAQM}(T)$.

Error rates. It’s well-known that a quantum algorithm with G gates can succeed if each gate has an error rate of $O(1/G)$, and this is often a necessary condition as well (with even more stringent conditions if idle qubits are also error-prone). The error rates in current quantum devices are too high for circuits with more than a few hundred gates, and experts seem to generally expect the future to centre around fault-tolerant error-correcting codes, not spectacular jumps in physical error rates [85].

For this reason, if a QRAM technology requires error rates per *component* to scale inversely with the number of bits of memory we do not consider this an effective approach to passive hardware-QRAM, regarding such components as just as unlikely as quantum computing without error correction.

Where this might not matter is applications with only a small number of QRAM accesses. As

an example, if an algorithm needs 10^6 Hadamard gates but only 10^2 QRAM accesses, then physical error rates of 10^{-3} are too high for Hadamard gates but just fine for the QRAM. In this case we would not need error correction, and in fact we could decode a logical state to a physical state, apply the QRAM, then re-encode to the logical state. This is a risky process for a quantum state, but if we make few total accesses to the QRAM then we also do few total decodings. This might be helpful, but we stress again that this limits the potential total size of the QRAM.

Bucket-brigade QRAM specifically only needs per-component error rates to scale inversely with the *log* of the number of bits of memory [59]. The overall QRAM error cannot be less than the physical error rates, so this is not fault-tolerant at arbitrarily large scales, but we discuss how this might reach practical scales in Section 9.4.

For the large scales of QRAM which must be error-corrected, we consider two approaches: error correction within the QRAM, or QRAM within the error correction.

7.1 Error-corrected components

Suppose that we attempt to suppress errors in each component of a QRAM device using some form of error correction, such as [35] propose for their method: “scaling up the qRAM necessitates further exploration in converting each tree node to a logical qubit and adapting quantum error correction.”

If this error correction requires active intervention, like a surface code, we have essentially recreated circuit QRAM. Even if the QRAM device uses specialized hardware, active error correction implies intervention from the controller and energy dissipation for each component, pushing us back into the regime of active hardware-QRAM with a $\Omega(N)$ cost of access.

Passive error correction, where the system naturally removes errors by evolving to lower energy states, is dubious. There are no known methods in 2 or 3 physical dimensions and there are several broad impossibility results (see the survey in [27]). Methods for passive error correction would be a massive breakthrough, but it would also not be enough for passive QRAM: passive QRAM also needs passive gates on the error-corrected components (see Section 6.1).

Because of these difficulties, it seems infeasible

to build error-correction within the QRAM that does not lead to access costs at least proportional to N . Thus, to avoid strict error requirements, we want a QRAM gate such that the total error of each access relative to the error rate of each component is only $O(\text{polylog}(N))$. Using “error” imprecisely to give a sense of scale, if $N = 2^{50} (\approx 10^{15})$ and the error scales as $\log^2 N$ as in [59], we “only” need error rates of 4×10^{-7} in each component to achieve a total error rate of 10^{-3} .

7.2 Error Correction

Instead of correcting the errors within the components of a QRAM device, we could instead attempt to correct the error-affected output of a noisy device. Because the set of possible QRACM gates includes the CNOT and Toffoli gates, the Gottesman-Knill theorem means we cannot expect to apply QRACM transversally to any error-correcting code for which Clifford gates are also transversal (such as a surface code). Here we imagine a few different non-exhaustive approaches:

1. apply QRACM gates independently of the code and attempt to inject the results into another fault-tolerant quantum system;
2. construct an error-correcting code for which QRACM is transversal, and inject *other* gates into this code as needed;
3. code switch from a more practical code to a QRACM-transversal code;
4. find a non-transversal method to fault-tolerantly apply QRACM gates.

We are pessimistic about QRAM error correction, for reasons we discuss below, but this area needs more research. Finding constructions for satisfactory codes, or proving more impossibility results, would shed more light on the feasibility of QRAM.

Transversal codes. There are a few issues with a QRACM-transversal code.

The transversal gates for a code form a group. Once we have a transversal QRACM of at least 2 bits of input, if the memory does not encode a linear function, we can fix certain inputs and precompose with X gates to construct a Toffoli gate from the QRACM. Once we have transversal Toffoli gates, we can compose them to construct all

possible QRACM, or more generally all classical reversible circuits. Thus, transversal QRACM is an “all-or-nothing” situation for a code.

Having all possible QRACM gates transversal for a code is so powerful that it is likely impossible. Almost all codes are proven to fail at this [86]. This would also contain every level of the Clifford hierarchy, since the n -multi-controlled NOT is in the n th level and can be realized with QRACM (consider access to a table T where only the $N - 1$ entry is 1, the rest are 0). Using [91, Theorem 2], this means QRACM cannot be transversal for any family of subsystem codes that can correct any probability of qubit loss.

If such a code were to exist, it would also be unwieldy for computations, as it will require another procedure like state injection to perform H gates (since Toffoli+ H is universal). Switching from a more practical code to a QRACM-specific code would need a series of intermediate codes to preserve the logical state, lest it decohere during the transition.

To escape these no-go theorems, we could use a different code for the output qubit, so that we cannot compose the transversal gates. Using this strategy allows quantum Reed-Solomon codes to have transversal Toffoli gates, though extending the method to n -bit QRACM implies the output qubit needs to be encoded in 2^n physical qubits.

7.3 Gate Teleportation

Universal quantum computation in a surface code is possible via magic state distillation and gate teleportation. Several auxiliary qubits are prepared in a fixed state, the controller applies a physical T gate to each one, then distills them together with measurement to create a much higher fidelity state. Using this one state, the T gate can be teleported onto any qubit in the surface code.

For an analogous process with QRACM, we define the following gate teleportation channel, shown in Figure 7. Let $\mathcal{H}_Q$ be the address and output space of the QRACM, and let $\mathcal{Q}(T) : B(\mathcal{H}_Q) \rightarrow B(\mathcal{H}_Q)$ be a channel representing the physical QRACM gate. We assume (using $\circ$ to mean function composition) $\mathcal{Q}(T) = \mathcal{N}_1 \circ \mathcal{U}_{QRACM}(T) \circ \mathcal{N}_0$ for all T , meaning that our QRACM process is modelled as a perfect QRACM access sandwiched between two noise channels. We model in this way so that the noise channels are not dependent on the table itself,

which aids in the proofs.

We have d distillation channels $\Phi_1, \dots, \Phi_d$, with $\Phi_i : B(\mathcal{H}_{i-1} \otimes \mathcal{H}_Q) \rightarrow B(\mathcal{H}_i \otimes \mathcal{H}_Q)$, and the entire state distillation process produces a state ρ_{distill} as follows:

$$\begin{aligned} \rho_{\text{distill}}(T) &:= \Phi_d \circ (I_{d-1} \otimes \mathcal{Q}(T)) \circ \\ &\quad \Phi_{d-1} \circ (I_{d-2} \otimes \mathcal{Q}(T)) \circ \\ &\quad \dots \\ &\quad \Phi_2 \circ (I_1 \otimes \mathcal{Q}(T)) \circ \Phi_1(|0\rangle\langle 0|) \end{aligned} \quad (14)$$

Letting $\mathcal{H}_{QL}$ represent the space of a logically encoded input and output space, we have a final teleportation channel $\Psi_T : B(\mathcal{H}_d \otimes \mathcal{H}_Q \otimes \mathcal{H}_{QL}) \rightarrow B(\mathcal{H}_{QL})$. We create a logical QRACM channel $\mathcal{Q}_L(T)$ by

$$\mathcal{Q}_L(T)(\rho) = \Psi(\rho_{\text{distill}}(T) \otimes \rho). \quad (15)$$

The goal is for $\mathcal{Q}_L(T)(\rho)$ to approximate a QRACM access on the logical state.

In the appendix we prove Theorem C.2, summarized approximately as:

Theorem 7.1. *Suppose a QRACM distillation and teleportation process uses d calls to the physical QRACM gate. Then the minimum fidelity of this distillation process is bounded above by $\frac{3}{4} + d\sqrt{\frac{2}{N}}$.*

That is, if we want a high fidelity QRACM operation to fit into a fault-tolerant computation, we must use resources that are asymptotically similar to those required to do fault-tolerant circuit QRACM.

The idea of the proof is fairly simple: considering the process in Figure 7, the input states to the QRACM are fixed. This means there is always some address j whose amplitude in the input state is on the order of $O(1/N)$. If we make a new table T' such that $T'[j] \neq T[j]$, then this is barely distinguishable to the state distillation process, since so little of the input to the QRACM will detect this difference. Yet, once we distill this state, it must suffice for all logical inputs. In particular, when our logical input is precisely the modified address j , the output between tables T and T' should be orthogonal. This is a contradiction: the distilled states are too close to create this orthogonal output.

A key component of this proof is that the state distillation is the same process for all tables, which is somewhat unrealistic. The specific pre-

and post-processing channels could be adapted to the specific table at hand. However, counting arguments similar to Theorem B.3 might still apply; using only $o(N)$ resources in the distillation could mean that a single process handles an exponential number of different tables, and the proof of Theorem C.2 will still suffice. Unlike Theorem B.3, measurements are necessarily involved in distillation and teleportation, so formalizing this line of reasoning would need a more sophisticated argument.

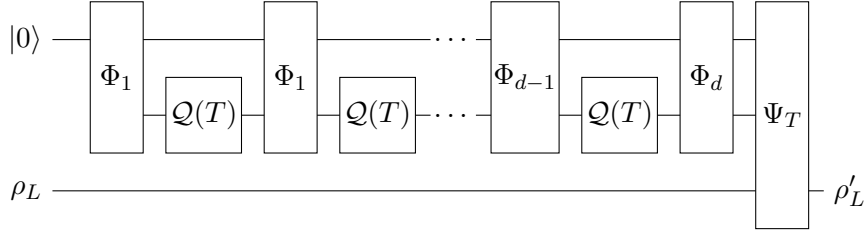
Theorem C.2 is a harsh blow to certain approaches to QRAM, but not all. It states that minimum fidelity is bounded, so applications with “broad” inputs to the QRAM, like database HHL and look-ups to classical pre-computation, are not yet forbidden. However, database-QAA suffers greatly. As the QAA hones in on the correct output, its input to the QRACM will get closer to this forbidden logical input. As a concrete example, if T and T' differ in precisely the address that QAA needs to find, then a distilled-and-teleported QRACM will obviously fail.

As a final note, since a QRAQM process implies a QRACM process by simply initializing the quantum memory to a classical input, Theorem C.2 forbids similar QRAQM distillation and teleportation processes.

Heralded QRAM. By “heralded” we mean any QRAM technology involving some measurement which indicates whether the QRAM access succeeded or not, typically also implying that, if it succeeded, the fidelity of the resulting access is very high.

Heralded processes are perfect candidates for gate teleportation because we can try as many times as we want to produce a single good magic state, then teleport it into the code. The impossibility of QRAM state teleportation is a serious blow to heralded QRAM. We cannot produce a resource state independently of the “live” data to which we want to apply the QRAM, so we need to instead input the real data into the QRAM, somehow ensuring that when the QRAM fails, this does not decohere the input.

Logical Inputs. Another potential route to save a process like this is to use the logical input as part of the QRACM access. This is more analogous to a transversal QRACM access, albeit with a very broad definition of “transversal” meaning only that we use a physical gate as part of the



$$\text{Goal: } \rho'_L \approx U_{QRACM}(T)\rho_L U_{QRACM}(T)^\dagger$$

Figure 7: Schematic of a QRACM disillation and teleportation. The channels $\Phi_1, \dots, \Phi_d$ attempt to distill the output of accesses to the physical QRACM table T , denoted $\mathcal{Q}(T)$. A final teleportation channel Ψ_T attempts to use the resulting distilled state to “teleport” the QRACM gate onto a logical state input ρ_L .

process to enact a logical gate.

This is a risky process, since the QRACM might noisily decohere the logical input. We want the full logical QRACM gate to have a much higher fidelity than the physical QRACM gate, so there must be little-to-no correlation between the logical state and the input to the QRACM. Intuitively, this suggests an extension of Theorem C.2 might exist, since if there is no correlation, then a similar table-swapping trick might work. We leave the details of a proof or disproof of this idea to future work.

8 Classical RAM

The extensive list of restrictions in Section 6 raises an important question of whether these same issues apply to classical memory. Here we apply the same arguments as Section 6 to classical memory. Either classical memory is fundamentally easier and escapes the restrictions of quantum memory, or the restrictions are already accounted in real-world analysis and use cases of large-scale classical memory.

8.1 Area-Time Metric

As Section 1.1 discusses, the central thesis of the memory peripheral model, and most critiques of QRAM, is that the qubits in the QRAM (and possibly the memory controllers themselves) could perform other computations more efficient than just the QRAM access. Succinctly, N bits of active QRAM is more like $\Theta(N)$ parallel processors ([103] states this explicitly).

This same argument applies to classical memory, and can mainly be justified by arguing that the cost to construct one bit of memory is proportional to the cost to construct one processor

([15, 16] cost algorithms in currency for this reason). While the ratio of costs might be quite high, it is constant relative to the input size of any problem, suggesting that the ratio of memory to processors should stay roughly constant. Some real-world examples of this phenomenon:

- Google’s “IMAGEN” [99], a huge text-to-image diffusion model, used 10 TB of public data plus an internal dataset of a similar size. The training used 512 TPU-v4 chips. Adding up all the independent processing units (i.e., arithmetic logic units (ALUs)), each chip has 2^{21} parallel “processors”, for a total of 2^{30} processors [54]. This means there is only about 18 kB of memory per ALU.
- GPT-3 used 570 GB of training data and used 175 billion parameters [28]. They use a supercomputer with more than 285,000 CPUs and 10,000 NVIDIA V100 GPUs. Each GPU has 2^{12} CUDA cores, totalling at least 2^{26} parallel processing units, giving 11 kB per processor.
- In the current largest supercomputer in the world, “Frontier” [38], each node has 5 TB of memory and 4 AMD Instinct MI250X GPUs. Each GPU has 14,080 stream processors [4]. This memory/processor ratio is much higher, but still not huge, at about 89 MB/processor.

While the individual processors in a GPU or TPU are not what we usually think of as a “processor”, they are capable of sequential computation. Machine learning benefits because it is a very parallelizable task, but many problems with algorithms using QRAM also have alternative parallel classical algorithms.

All of this favours the *area-time* cost model for classical computation, where the cost is the product of the total amount of hardware by the runtime of an algorithm.

A further motivation is parallelism, as in [22]. Many classical algorithms parallelize perfectly, like the machine learning training above, the linear algebra in Section 4, classical lattice attacks, and unstructured search. This gives little reason to favour a computer with a large amount of memory with few processors, since for only a constant factor increase, many processors can share that memory and achieve the same result faster. As [17] say about a chip with most of its components idle: “any such chip is obviously highly suboptimal: for essentially the same investment in chip area one can build a much more active chip that stores the same data and that at the same time performs many other useful computations”. More directly, someone paying for server time literally pays for the area-time product of their computation.

To summarize, the notion that the qubits and controllers in QRAM can be repurposed to more fruitful tasks is an argument that carries over directly to classical computing, and it is actually applied to large, high-performance computing for vast improvements in runtime with little increase in total cost.

8.2 Active Components

We argued in Section 6 that a QRAM device will probably need interactions with an external environment for the components to function properly. The same is true for classical memory, but here we spell out the biggest difference between quantum and classical memory: classical memory only needs $O(\log N)$ active components to access memory.

Here we say that a component is “active” if it is capable of performing whatever action it needs to properly execute a memory access, and assume that if it is inactive any memory access requiring that component will fail. As an example, in the SWAP-based circuit in Figure 4, each node is active only while the SWAP gates are applied to it. In the bucket-brigade proposal of [52] with atoms as routers, the atoms must be stimulated to be active. By our definition of passive vs. active QRAM, most of the components in an active QRAM will require external intervention to be

active.

We sketch here how a classical memory could work with only $O(\log n)$ active components: there is a binary tree of routers, all of which are powered off. Each router is capable of receiving a signal which it amplifies and uses to decide to start drawing power. To route a memory request, the router in layer i of the tree, once activated, uses the i th address bit to decide whether to forward the address to its left or right child node. The process then repeats.

In this way, only $O(\log N)$ routers are actually on. It is able to do this because there is no restriction about leaking access patterns to the environment.

Our concern for quantum memory is that if activating a router requires any interaction with the external environment, then that interaction cannot depend on the quantum state input to the memory, lest the environment become entangled with the state and decohere the input. Thus, for such an architecture, the environment (the control hardware) must decide, *independently of the input state*, which components to activate.

More specifically, if the input is a uniform superposition of addresses, all the routers in the QRAM must be activated: if one router is not activated, then any address request sent through this router will fail. Based on the size of the QRAM, there must be $\Omega(N)$ routers, so we need to activate $\Omega(N)$ routers for a successful QRAM access with a uniform superposition of addresses (unless the QRAM is passive).

Even in algorithms where the input superposition over addresses only has support on a subset of the QRAM memory cells, such as in the final queries in Grover database search, the control hardware must act as though the input could be a uniform superposition.

For instance, if the control hardware decides not to supply power to routers for certain addresses, any superposition query with support on those addresses will fail. To correctly handle all queries using only some of routers, the control hardware would need to identify which addresses are supported in the coherent quantum state and which are not. However, since the control hardware operates classically, acquiring this information would collapse the quantum state, thereby

although technically there would still be non-zero support on most states

invalidating the algorithm. Thus, to reiterate, the control hardware must treat this as though the input *could be* a uniform superposition.

Incorporating algorithm-specific analysis, the control hardware might know that there is a particular subset of M memory locations such that all possible input addresses will be in this subset (this happens in certain quantum lattice sieves [33, 73]). Then we can treat this as a QRAM access to only M bits (with a cost of $\Omega(M)$) rather than N , since we could construct a QRAM device that only accesses those bits. However, this relies on knowing exactly *which* locations will have zero support in any input state, and in most algorithms this is not known (and thus this would not be a general QRAM). For example, in a Grover database search, if we know that the desired element is not in certain addresses, we’ve already solved part of the search problem!

Example: Flash memory. Real-world classical memory devices do not precisely follow this low-power model. Following the description of [30], flash memory is divided into *chips*, which are divided into *dies*, which are divided into *planes*. Within each plane are thousands of *blocks*, each of which contains hundreds of *pages*, each of which contains thousands of bits. The device does not address individual bits, but full pages. Each of these components are integrated circuits, and mostly we do not care about their exact structure or function, since our concern is the fact that each one (chip, die, plane, and block) must have a mechanism to select specific subcomponents based on an address as input.

For example, near the bottom of this hierarchy, to access a specific block within a plane, two multiplexers (similar in function to a QRAM routing tree) decode the address of the block into a column and a row. A signal is sent along a wire at that column and at that row such that it activates the block at a memory location at the intersection of the selected column and row. From there, another multiplexer selects the appropriate page, and the device acts on that page (to read, write, etc.).

Abstracting this slightly, each block performs the equivalent of an AND gate: it returns the memory address only if there is a signal on the column bit line *and* on the row word line. Thus, as with QRAM, the number of “gates” is pro-

portional to the number of pages of memory; however, in contrast to QRAM, only the “gates” on the active lines consume energy. In other words, the energy consumption to address one block should be proportional to the size of the length of each column and the length of each row (approximately the square root of the total number of blocks), and experiment confirms this [82].

This experimental result implies also that the analogous process *within* the block, to address the page, consumes energy only in the blocks on the active row and column lines, at most. This is substantially fewer than the total number of blocks, each of which has its own page decoder, meaning that the page decoding process matches the abstract approach of only activating the addressed sub-unit of memory. Here the analogy stops: addressing a single page within a block sends different signals to addressed and non-addressed pages, so the energy per active block is proportional to the size of the block.

Above the block level, to achieve minimum energy consumption we would expect each die to only activate the necessary planes, and each chip to only activate the necessary dies. Instead, the exact opposite happens: flash SSDs will group blocks and pages on different chips and planes into *superblocks* and *superpages*, which are accessed simultaneously. The purpose is so “the SSD can fully exploit the internal parallelism offered by multiple planes/chips, which in turn maximizes write throughput” [30]. Here again we see the “repurpose memory into processors” maxim: SSD designers expect that any application needing access to the SSD will have enough processors to reap more benefit from fast parallel access than from a more energy-efficient design.

Active memory. Any volatile classical memory, such as DRAM, is analogous to active hardware-QRAM, in that the total energy cost for N bits is at least proportional to N . In fact, it’s even worse: volatile memory requires $\Omega(N)$ energy *per clock cycle*.

Despite this, DRAM sees widespread use. For consumer-level computers, RAM is often not the highest energy component, consuming less than any other component in an idle smartphone [31] and similarly for laptops. Though the asymptotic cost might increase with memory size, at practical sizes this is irrelevant. This cost might matter for large-scale applications; however, as in Sec-

tion 8.1, in practice other components are also scaled up for these applications, so the energy draw of the RAM is not the main cost. Again, the area-time model captures this: total power draw increases with both the amount of volatile memory and the number of processors, so efficient machines will scale both proportionally.

8.3 Error Propagation and Correction

In Section 6 we claimed that QRAM technologies that require error rates to scale as $O(1/N)$ per component will probably not be useful. One could make the same argument that if a single gate at one of the row and column lines of DRAM or flash memory spontaneously emits current, then it will create an error in the RAM. Thus, we should expect classical RAM to need component error rates to also scale as $O(1/N)$. Why is this acceptable classically?

The first reason is that extremely high fidelity classical components are fundamentally much easier to produce. Classical components have essentially one dimension of error: bit flips. In contrast, quantum components suffer from analogous bit-flip errors, but also phase-flip errors. Two-dimensional passively corrected memory, which is provably impossible for quantum memory except in esoteric settings [27], is easy and ubiquitous classically (viz. hard disk drives).

While this analogy holds for non-volatile memory, volatile classical memory has a further advantage from the continuous energy flow to help correct errors. The technique of a “cold boot attack” (such as [56]) highlights that volatile DRAM is somewhat non-volatile, but it just has an unacceptably high error rate unless power is continuously applied. Recall that we cannot apply power continuously to a passive hardware-QRAM, or else it becomes an active hardware-QRAM. Again, if a QRAM device *does* pay this cost, such as circuit QRAM, it could work perfectly well, but many applications (e.g. database HHL and database QAA) lose their advantage.

Second, classical memory can, and does, perform error correction. Circuit QRAM is allowed to require $O(1/N)$ error because it can be part of an error corrected quantum computer that achieves that level of noise with polylogarithmic overhead. Beyond the consumer level, classical memory *also* uses error correcting codes because the overall failure rate is too high. The error

rate for RAM, in failures per Mbit per million hours of operation, is somewhere around 25-75 failures [101] or 0.2 to 0.8 [116], though with high variance not only between devices but also over time for a given device.

This error rate shows two points: first, it is low enough to be negligible for most consumer applications, and second, it is still high enough to be relevant at large scales. This exactly matches our claims about QRAM. The classical difference is that the achievable error rate for physical components is much, much smaller.

Finally, errors present less of a problem for classical computing because of the no-cloning theorem. Modern large-scale, high-performance computing often follows the “checkpoint/restart model” [50], where the computer periodically saves the entire state of memory so that it can resume in the case of a fault. A quantum computer cannot do this because of the no-cloning theorem. Instead, the error rate of each operation must be driven so low that the *entire* computation can proceed without fault.

8.4 Latency

All memory devices, classical and quantum, must contend with signal latency and connectivity. If connections between qubits are limited to fixed length, then because we live in Euclidean space (a) each qubit can have only a constant number of connections, and (b) sending data between qubits on opposite ends of the memory requires at least $W^{1/d+o(1)}$ connections in d -dimensional space, if there are W total qubits. Long-range connections still suffer from signal latency, where – asymptotically – the time for signals to propagate across a memory device is still $W^{1/d+o(1)}$.

We emphasize that even light-speed signals have noticeable latency: for a 3 GHz processor, light travels only 10 cm in each clock cycle. We often don’t see this, because (again) huge classical data structures typically co-exist with enormous numbers of parallel processors (as [22] predict based on this reasoning). Classical memory latency has led to tremendous effort in efficient caching and routing. In contrast, quantum memory access “takes the mother of all cache misses” [100], since any timing information effectively measures which path the access took, and hence which address.

Thus, the case where latency would be an issue

– where a single processor attempts to process an enormous amount of data with completely unpredictable queries – is extremely rare for classical computers. Yet applications like database HHL typically assume a single quantum processor, and even when assuming parallel access like [13], the quantum memory cannot efficiently cache. Overall, quantum memory latency seems more difficult to manage than classical.

9 Bucket-Brigade QRAM

The bucket-brigade model of QRAM [52] is the most ubiquitous. We describe it here and discuss various proposals to realize the bucket-brigade and a recent result on its error rates.

9.1 Construction

The original proposal differed slightly from the circuit version we showed in Section 5.3. It referred to “atoms” and “photons”, and we will continue to use this terminology, though any physical system with similar properties will suffice.

As in the circuit case, we use a binary tree of nodes to route incoming signals to the desired memory cell. However, each node is now an atom, and the lines between nodes represent paths for a photon. The atoms have three states: $|W\rangle_A$ (“wait”), $|L\rangle_A$ (“left”), and $|R\rangle_A$ (“right”), and are initialized in $|W\rangle_A$. The photons have two states, $|L\rangle_P$ and $|R\rangle_P$.

For QRAM access, we generate a photon from each address qubit (from most significant to least), where $|0\rangle$ generates $|L\rangle_P$ and $|1\rangle$ generates $|R\rangle_P$. When the photon reaches an atom, it interacts depending on the state of the atom:

- If the atom is in the wait state $|W\rangle_A$, the photon is absorbed and transforms the atom to $|L\rangle_A$ or $|R\rangle_A$, depending on whether the photon was in $|L\rangle_P$ or $|R\rangle_P$.
- If the atom was in the state $|L\rangle_A$, the photon is deflected forwards along the **left** path, with no change in atomic state.
- If the atom was in the state $|R\rangle_A$, the photon is deflected forwards along the **right** path, with no change in atomic state.

Sequentially sending one photon for each address qubit will create a path in the tree of nodes to the addressed memory location.

The intended feature of this architecture is that *if* we can produce a physical system with these properties, then the entire system needs no external intervention besides inputting photons. We’re meant to imagine something like the Galton board or Digi-Comp II [53], wooden devices where balls enter on one side and propagate through with momentum or gravity. These are all *ballistic* computations.

Recall that if we must intervene at every step, then the system is no longer ballistic and loses many of its advantages. For example, in their original proposal, [51] suggest stimulated Raman emission for the atom-photon interactions; however, *stimulating* that emission with an external field requires work from the control program and/or environment. This implies a cost growing proportional to N , which is within a constant factor of circuit QRAM!

This is not the only way to construct a bucket-brigade QRAM. For example, the atoms could have just the two states $|L\rangle_A$ and $|R\rangle_A$ and be all initialized to one direction, as in [59]. Their description requires a swap between the incident photon and the node atoms, which requires some active intervention, since (as in the circuit case) we will need to intervene to change the behaviour from swapping into the node atom or propagating through. Hence, two-level photons with two-level atoms is inherently an *active* hardware-QRAM.

These are not exhaustive examples, but give some flavour for how bucket-brigade QRAM could be realized.

9.2 Reversibility

A passive bucket-brigade QRAM will need to evolve under a single Hamiltonian, and thus each transition must be reversible. This creates some challenges, since the intended function of a node atom is that once it transitions from the wait state $|W\rangle_A$ to either the left or right states, it should stay in those other states. However, this is not reversible. More plausibly, the left and right states will spontaneously re-emit the photon that excited the wait state. In fact this behaviour appeared from the mathematics of a more complete Hamiltonian analysis in [29].

This could be a feature, however. We also want

to restore the QRAM nodes to their idle state after each access, so if the nodes are carefully engineered so that the final layer’s relaxation time is smallest, they will spontaneously re-emit their excitations in reverse order, neatly sending all the control photons back through the routing tree to the address register. Ensuring this happens is an often overlooked problem of passive hardware-QRAM.

More generally, even a simplistic analysis of reversibility creates problems. Considering each node, for each of the three output lines there are 5 states to consider: no photon or a {left,right} photon propagating {up,down}. For the atom, there are three states (possibly more if we imagine a “timer” of how long until an excited atom re-emits its photon). This means each node could be in one of 375 different states, and the QRAM Hamiltonian must act reversibly on them. The necessary functions for routing constrain this only slightly.

For example, if a $|0\rangle_P$ photon propagates upward from one of the child nodes and reaches an atom in the wait state, what should happen? If it continues propagating upward we lose the favourable scaling proven in [59]. Instead, if it is reflected backwards, or to the other side, then an erroneous spontaneous emission at some point in the tree will stay constrained to oscillate back and forth among at most three nodes.

A final note on this is that if we must actively reset the state of all the nodes in a bucket-brigade, this has cost $\Omega(N)$, pushing us back to an active hardware-QRAM. If we do not reset the nodes actively, they must reset themselves. If any errors occur in this process, they will persist to the next QRAM access.

9.3 Proposed Technologies

Trapped Ions A particular architecture for bucket-brigade QRAM, proposed in [51], is to use trapped ions as the nodes, which would deflect and absorb photons as desired. However, this raises two problems.

The atoms must be carefully positioned to ensure the deflected photons travel between the two atoms. Thus, they must be fixed in place somehow. Earnshaw’s theorem forbids a static electric field from trapping a charged particle, so ion traps require continuously changing fields, such as a quadropole ion trap. Continuously changing

the field requires a constant input of energy.

Second, we need extremely high probabilities of absorption and deflection for the QRAM properly. With, e.g., 2^{40} memory elements, we would need error rates of something like $\epsilon/40^2$ for an overall error of ϵ in the QRAM access, since there are $\binom{40}{2}$ atom-photon interactions for even a single address. This seems implausible for trapped ions. [51] solve this with stimulated emissions, but as mentioned above this is no longer passive.

If this interaction problem is solved, the QRAM routing tree looks like a full-fledged trapped ion quantum computer. Each node has a trapped ion with spectacularly low error rates and a dedicated high-precision laser or microwave emitter. The ions can communicate with each other by the same photons used for routing memory addresses.

[62] propose microtoroidal resonators instead of trapped ions, but they have the same issue of requiring external photons to drive the interactions at each node.

Transmons Breaking from literal atoms and photons, [29] proposed a bucket-brigade where the “atoms” consist of 3 superconducting transmons. This provided enough parameters (frequencies, capacitance, etc.) to precisely tune for a high certainty that photons will be absorbed and routed as desired. Their proposed device is truly passive and the desired photon transmission probabilities are over 95%.

They mention a critical flaw in their technique which prevents it from scaling. The first photon, which modifies the state of the transmon from “wait” to “left” or “right”, must be at a different frequency from the second photon, which will be routed left or right. This means once the second photon is routed, it cannot modify the state of another device. In other words, the routing tree can have at most one layer.

Solutions might exist. They propose increasing the frequency of the photon, though this requires external energy input to the device and must be done without learning which path the photon took while not scaling energy proportional to the number of transmons, a great challenge. Instead, one might also use slightly different parameters in each layer of the tree, so the frequency which excites nodes in layer i is the same frequency that layer $i - 1$ will route. The precise approach in

[29] does not allow this; the range of plausible parameters forbids such layering.

Finally, while the transmission probabilities are high for near-term devices, they are insufficient to scale to a large memory device or a high-fidelity application.

Despite these issues, this is a promising approach, as it explicitly accounts for the fundamental issues in passive hardware-QRAM.

Photonic transistors [111] constructed a photonic transistor with transmons in optical cavities. This could take the place of the node “atoms”, but the photon is not absorbed by the transmon and instead resonates with it in the cavity. In a bucket-brigade, the photon from one bit of the address would need to resonate with all the nodes in one level of the tree, which a single photon could not do coherently.

Heralded Routers [35] propose a “heralded” version of bucket-brigade QRAM. The heralded aspect is that they use a photon-atom interaction wherein the photonic state is teleported into the atom, so the device must measure the photon. If the measurement detects no photon, then the photon was lost due to, e.g., propagation error, and the QRAM access can be restarted. Crucially, this avoids the issue of heralding described in Section 7.3, since measuring photon loss reveals only that a photon was lost, not *which* photon was lost.

A crucial design choice is that all routers in a single layer of the routing scheme must share their measurement of the photon. Measuring at each detector would immediately reveal the routing path and decohere the address state, so instead we must replace the detectors each each router with waveguides that all converge at a single detector. Similarly, they require a method to simultaneously measure the spins of $\Omega(N)$ atoms; they propose an ancillary photon for this. Altogether this will lead to one measurement per layer of the routing tree, so the controller only needs to perform $O(\log^2 N)$ operations.

As written, their design requires a Hadamard gate on the atom in each router to perform the oblivious teleportation described above, which means $\Omega(N)$ active operations to perform a memory access. However, they also need the atoms initialized in the $|+\rangle$ state to obliviously teleport

the photonic input states onto the routers’ atoms. Thus, they could skip both operations by somehow simultaneously measuring the spins of all the atoms in the $\{|+\rangle, |-\rangle\}$ basis. Since only one atom will not be in the $|+\rangle$ state, measuring the parity of all the spins will perform the necessary teleportation if no errors have occurred.

If this measurement detects an even parity, then no correction must be done and all atoms are initialized for a subsequent read; however, odd parity means one atom, at an unknown position which is correlated with the address register, is in the $|-\rangle$ state and must be corrected. This is problematic: we now need a second process or device to correctly route a signal to correct that atom, without ever classically learning where the signal is going. Perhaps this could be accomplished with another photon sent through the system (which is already set to route photons to the atom in the $|-\rangle$ state).

Thus, there are some missing components for this to fully realize a passive hardware-QRAM. Moreover, it suffers from the same ion trap QRAM issues of needing high-probability atom-photon interactions and passively trapped ions.

[35] also propose teleporting the address into the nodes of the tree before the routing process, which requires $\Theta(N)$ gates applied to the device and thus fails to realize passive hardware-QRAM.

Photonic CNOT. Another heralded QRAM is [113], who use a post-selected photonic CNOT for their quantum router such that each routing operation has, at best, a 0.0727 chance of failure. This means we can apply at most 9 routers sequentially before we have only a 50% chance of success. Bucket-brigade QRAM with n layers requires $\binom{n}{2}$ routing operations, meaning 9 routings is only about 5 layers (i.e., a 32 bit memory). However, because their scheme requires the control and signal photons to enter the router simultaneously, we may need several routers in each layer to simultaneously route more address qubits. Ultimately, this approach doesn’t scale.

9.4 Error Rates

An advantage of the bucket-brigade approach – even as a circuit-based QRAM – is favourable error scaling, shown by [59]. To give an intuition of their argument, in any circuit-based QRAM, there are $\Omega(N)$ gates and so we might expect

each gate to need error rates less than $O(1/N)$. This is true for unary QRAM, which one can see with a simple analysis. However, with the bucket brigade, errors can get stuck. That is, if an error occurs on some node deep in the tree, this does not necessarily affect any other nodes.

The only way for errors to affect the final result is if they propagate to the root node somehow. At each node, the routing circuit only swaps from one of its two child nodes, so the error will only propagate upward if it is on the correct side. As there is only one path which will get routed all the way to the top of the tree – the path representing the actual address in the query – an error propagates to the root only if it occurs on this path. But only $O(\log N)$ qubits are on this path.

Errors could propagate in other ways; for example, if the control qubit has an error *and* the routing qubits have errors, then this will erroneously create a path that propagates further up. But for the error to keep propagating upward, a very precise sequence of errors must occur to create a path for it, and this is unlikely for even modestly low probability errors. Overall, if qubit undergoes an error channel with probability p , the error of the final readout is only $O(p \cdot \log^2(N))$ [59].

The bucket-brigade requires all nodes to be in the “wait” state at the start of an access. However, errors in a previous access that change this state could persist to later accesses, depending on the device’s Hamiltonian. If the probability of a persistent error is p for each node in each access, then after Q accesses the proportion of error-ridden nodes grows proportional to $Q^2 p$.

Generally, the per-component error rate p must still be exponentially small for applications like database QAA, but for other applications, like quantum linear algebra (see Section 4.4), higher error rates could be manageable.

Moreover, we propose that a bucket-brigade QRAM system could be *partially passive*. Following the observation that errors in the root nodes of are more destructive, we can run full error-correction on the first k layers of the bucket-brigade tree. If k is selected to be a constant, or at least $o(\log N)$, then this circumvents the issues with active QRAM that we have extensively highlighted. The remaining $\log(N) - k$ layers would be a collection of smaller, passive

QRAMs joined to the error-corrected root nodes. This encounters the same problem of interfacing noisy components with quantum data encoded into a logical state that we discuss in Section 7. This proposal requires 2^k error-corrected nodes, and we expect that the overall probability of error would scale something like $O(p(\log(N) - k)^2)$. This wouldn’t change the asymptotic error rate of bucket-brigade, but with the same physical error rate, this method could allow larger memories to become practical than a fully passive bucket-brigade architecture would allow.

Example. To show why the small bucket-brigade error rates are somewhat unique, we describe here an alternative readout circuit that *does* require $O(1/N)$ gate error, as shown in Figure 8. To explain this circuit, we add an extra layer L of qubits at the leaves of the routing tree (shown as boxes in Figure 8). We then add a single extra qubit prepared in the $|1\rangle$ state, and propagate it through the tree by applying the routing circuits at each node. This routes the $|1\rangle$ to the location indexed by the address register. From there, for every i such that $T[i] = 1$, we apply a CNOT from the leaf at the i th address to a qubit in L . This means this final layer of qubits is all zeros if $T[i] = 0$, and has precisely one qubit in the $|1\rangle$ state otherwise. Thus, we can read out the result by computing the parity of this final layer.

Parity is an appealing operation in the surface code, since it can be done by conjugating a multi-target CNOT (itself depth-1 in a surface code) by Hadamard gates.

The drawback of this method is that if an error occurs anywhere on this final layer of ancilla qubits, it will propagate immediately to the output. Thus, the qubits and gates in L would need to have error rates of $O(1/N)$.

10 Other Proposals

Here we summarize some other proposals which do not follow the bucket-brigade approach, and detail the ways in which they fail to provide passive hardware-QRAM, as they are currently described.

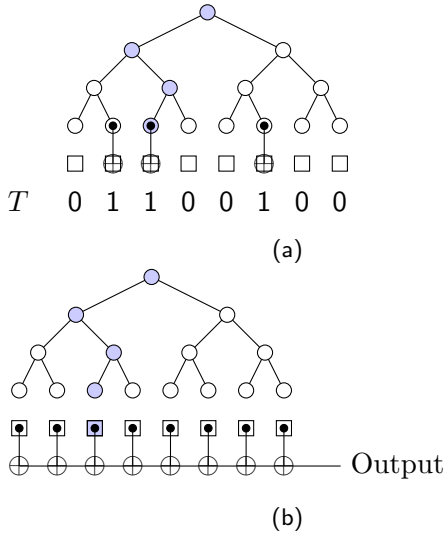


Figure 8: Alternative, suboptimal bucket-brigade read-out forcing the readout gates to have error rates of $O(1/N)$. The blue nodes are those with $|1\rangle$ during the memory access for a specific address (in this case, $i = 2$). Notice that the parity in the second step only needs to use leaves for i with $T[i] = 1$, but we include all of them for clarity.

10.1 Time-bins

Many applications and methods to store quantum states with photon echoes exist (e.g. [109]), but [83] propose an additional addressing component to turn this storage into QRAM. To summarize their method, there is a memory cavity with $M \gg N$ atoms and a control cavity. To store data, the device would send quantum states encoded as photons into the memory cavity, where they would excite the memory atoms. The memory atoms can replay this input via a two-pulse echo technique; sending a second pulse will prompt the atoms to re-emit the photons that were sent in. Importantly, if photons are sent in one-at-a-time, they will be re-emitted in a time-reversed order.

Thus, to read the memory, the device sends a pulse which prompts the memory cavity to re-emit its stored state. With just the memory cavity, this would re-emit all input states at classically determined times, giving us CRAQM but not QRAQM. For quantum addressing they use a second *control* cavity. The control cavity contains an atom such that, when this atom is in its ground state, the memory cavity cannot emit any photons. However, when the control cavity's atom is excited, it allows the transfer of photons out of the memory cavity.

An address state $\sum_i \alpha_i |i\rangle$ must then be translated so that each address $|i\rangle$ is entangled with a photonic state $|\psi_i(t)\rangle$ which reaches the control cavity at precisely the right time to allow the memory cavity's photons to pass through. In this way, other addresses, which will not be entangled with a photon pulse at that time, will not allow any emission from the memory cavity.

The fundamental nature of the time-bin addressing means that this requires $\Omega(N)$ time for each memory access. The purported feature of this method is the minimal hardware requirements (though it still requires $\Omega(N)$ atoms in the memory cavity), but unary circuit QRAM already provides a near-optimal circuit on the $\Omega(N)$ side of the time-hardware tradeoff of QRAM circuits.

In [88], they use a second type of pulse to decohere a memory atom so that it will only re-emit once an identical pulse is sent to it. Using different frequencies of pulses to store different memory bits, this could reduce the total time requirement, at the expense of needing higher precision in pulse frequencies.

There is still an issue of how the time-bin photonic address state is created. As an example of how this could be created, consider the unary circuit QRAM. It classically iterates through addresses j , and for each address j , flips the state of a control qubit if the address in superposition equals j . Suppose that flipping the control qubit puts it into an excited state which can be stimulated to emit a photon. If we do this sequentially, then at time t_j , only the address state $|j\rangle$ will cause a photon to be emitted, thus the presence of photons at that time is in superposition and entangled with the state of the address. In this way, applying this stimulation instead of a CNOT turns the unary circuit QRAM into a circuit that translates an address superposition into a superposition of time-bin photons.

Unfortunately, this has solved the addressing problem by using another QRAM circuit to create the address! While superpositions of photons at different points in time *may* be a native quantum state in some architecture, for other architectures, creating such a state is essentially a quantum addressing problem, which is the difficult problem of QRAM anyway. The decohering pulses of [88] face a similar issue, where they must be addressed and sent in a coherent superposition

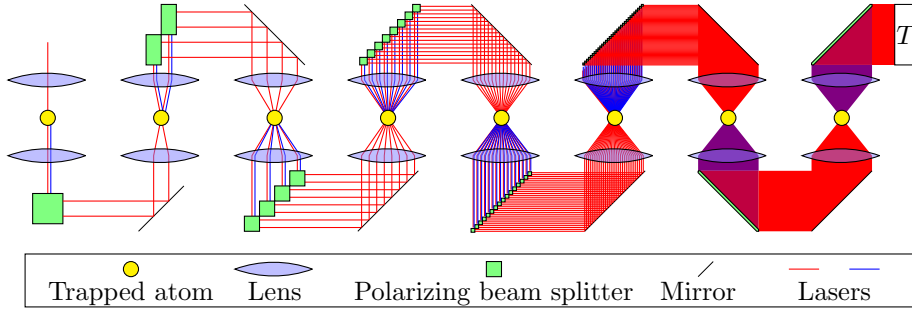


Figure 9: Schematic of quantum optical fanout QRAM, almost exactly as shown in [51].

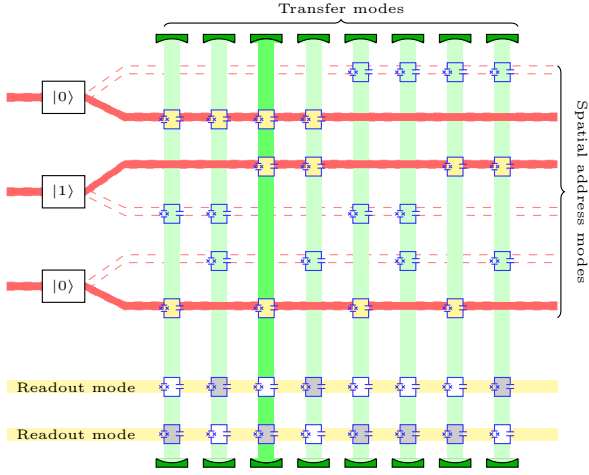


Figure 10: Phase gate fanout QRACM, from [51]. The address qubits route the address modes (red), which excites the phase-control qubits (blue transmons). The excited phase-control qubits are highlighted yellow. This will cause exactly one cavity's transfer mode to resonate (dark green), exciting the 2 memory qubits (bottom, blue) for that address. The memory qubits change the phase of the readout modes (yellow) which can be detected. The arrangement of the two types of memory qubits (shown as white and grey) represents the data: in this case, the table is 01010001.

to create a QRAM device.

As a final note, many aspects of the scheme in [83] require $\Omega(N)$ active interventions. For example, with the photon echo technique, stimulating the memory cavity to emit its states requires a phase-flipping pulse on all M atoms for each time bin, meaning $\Omega(N)$ pulses that each require $\Omega(N)$ energy. As in other proposals with photon-atom interactions, the control cavity requires a stimulating pulse for each interaction (thus, another $\Omega(N)$ stimulating pulses).

10.2 Quantum Optical Fanout

[51] describe an architecture using only $\log N$

trapped atoms, and instead encode the QRAM access in N spatial modes of light. The basic principle is this: each bit of the address becomes one trapped atom, and then each beam interacts with the atom, and if the atom is in $|1\rangle$, the photon is polarized differently than if the atom is $|0\rangle$. Then the photon beam is sent through a half-wave plate which spatially separates different polarizations, thus doubling the number of photonic modes and ensuring that an input beam is directed according to the state of the atom.

Figure 9 duplicates Figure 4 from [51], with a crucial difference: we show the schematic for an 8-bit address, rather than only 3. This immediately highlights the problem: the number of spatial modes grows exponentially with the number of address bits, but they must be focused back onto a single point.

If the photons are not aimed precisely, then after interacting with the atom they will drift into another spatial mode. This causes an error. The precision required for this aiming is $O(1/N)$ for N bits of memory. This seems to be the only physical mechanism for errors to propagate from one sequence of spatial modes to another, suggesting that the overall error scaling has similar robustness to bucket-brigade QRAM, but the aiming argument above means that each component requires a precision scaling as $O(1/N)$ or it will have an error. Intuitively, it doesn't matter whether errors propagate across "paths" of access, if all paths have an error.

Trying to guide the photons with a waveguide does not fix the problem, regardless of the miniaturization we achieve. Taking the perspective of the atom, the sources and receptors of the photons and must occupy non-overlapping sections of all radial paths out of the atom. If either source or receptor has a smaller cross-sectional area, then we can move it closer, but it occupies

the same “arc-area” from the atom’s perspective. This means the precision of the beam must be the same – $O(1/N)$ – no matter the absolute area of the photonic components.

Thus, in the exact formulation given, errors in this QRAM must scale infeasibly. We can wonder whether a different technology, inspired by this approach, could succeed. A core function of this approach is that the i th address bit interacts with 2^i spatial modes in one “operation”. This is the crux: finding a mechanism to enact such a large number of interactions passively, with error scaling much less than 2^i , seems difficult if not impossible.

10.3 Phase Gate Fanout

[51] describe another quantum fanout system, shown in Figure 10. This system translates each address qubit into a photon in one of two spatial modes (based on $|0\rangle$ or $|1\rangle$ in the address qubit, shown as red horizontal beams in Figure 10). Each of these $2 \lg N$ spatial address modes couples to N phase-control qubits (the upper blue transmons in Figure 10), so that the phase-control qubits change state when there is a photon in this spatial address mode.

Each phase-control qubit also couples to another optical mode, which we’ll call a transfer mode (the vertical green beams in Figure 10), such that there are N transfer modes, each coupled to $\lg N$ phase-control qubits. The coupling is such that if the phase-control qubit is in its $|1\rangle$ state, it adds a phase to the transfer mode. This two-mode coupling is similar to the recently experimentally-realized photon transistor in [111].

The net effect of all this is that if the address is $|j\rangle$, then the i th transfer mode gains a phase of $e^{ij\varphi_0}$ for some constant φ_0 .

For readout, we have another collection of N memory qubits (the bottom two rows of transmons in Figure 10), each coupled to a different transfer mode, so that the j th memory qubit interacts with the transfer mode only when the transfer mode has a phase of $e^{ij\varphi_0}$. Thus, the memory qubit changes state depending on whether the transfer mode is in resonance. For the final readout, we use another optical readout mode (the yellow beams in Figure 10), which interacts with all memory qubits, and picks up a phase for each one that is active. This means we

will actually need $2N$ memory qubits and 2 readout modes, so that *which* readout mode gains a phase will encode the binary result of the memory lookup.

There are a number of practical issues with this scheme.

The foremost issue is the narrowness of the resonance of the memory qubits. [51] note that there will need to be N different resonant phases, and analyze the width of a Fabry-Perot cavity’s resonance and conclude that if the transmissivity of the beamsplitters in the cavity is $O(1/N)$, the width of resonant phases is also $O(1/N)$. However, this does not address the need to fabricate each cavity, where the resonant phase must be constructed with $O(1/N)$ precision as well. For a Fabry-Perot cavity, this would mean $O(1/N)$ precision in the physical distance between the mirrors.

Second, each address mode must interact with N qubits. Modelled with a Jaynes-Cummings Hamiltonian, every excited qubit means a reduced photon number in the address mode. If not all qubits are excited, we will end up with some superposition where only a small portion of the total amplitude corresponds to states where the *right* qubits are excited to properly perform the memory access. Thus, we need all qubits to be excited, meaning the photon number of each address mode must be $\Omega(N)$.

In addition to forcing us to use $\Omega(N \log N)$ energy per QRAM access, we must also provide this energy to the address modes in such a way that, for each address qubit, we do not learn *which* of the two address modes took this energy. If we learned this, it effectively measures the address qubit. The quantum router of [29] might suffice.

This QRAM may not even be passive, in fact. To excite the memory qubits, we do not want to activate a new pulse – which requires $\Omega(N)$ interventions and $\Omega(N)$ energy – but rather we want the memory qubits to reside in an optical cavity that maintains some electric field. Consider a back-of-the-envelope analysis of the losses of the optical cavities. If each cavity has quality factor Q and resonant frequency f , the fraction of energy lost per unit time is proportional to Nf/Q , since there are N cavities. Modern “ultra-high” quality factors are on the order of 10^9 [75], about the same scale as the frequency (in hertz) of microwaves. Thus, to maintain the

QRAM for more than a few seconds requires either enormous optical cavities (to increase the resonant frequency) or spectacular improvements in optical cavity quality factors. In either case, if these improvements do not keep pace with the increase in the number of memory bits, the *total* energy loss quickly becomes enormous.

Finally, each readout mode interacts with N qubits. If our error model permits some probability ϵ of adding the wrong phase from each memory qubit (e.g., the probability from measuring after interacting with that memory qubit), then we need ϵ to be at most $O(1/N)$, or the memory readout will fail.

Unlike [51], the arrangement of phase-control qubits in Figure 10 means that for each address, only one cavity has all n phase-control qubits excited. Potentially one could engineer each phase-control qubit to contribute $1/n$ of the necessary phase to bring the transfer mode into resonance. If so, the precision of the resonator might only need to be $O(1/\log N)$. However, given the readout method, any leakage from the resonators will contribute directly to readout error (if leakage excites the incorrect memory qubit, the readout detects this). Hence, the transfer modes may still need $O(1/N)$ precision.

10.4 Derangement Codes

[58] proposes error correction based on code switching, which we summarize and slightly tweak here.

We start by imagining a no-op state $|\text{nop}\rangle$, such that not only is $|\text{nop}\rangle$ invariant under U_{QRAM} , the internal quantum state of the QRAM nodes are also invariant when we input $|\text{nop}\rangle$. As an example for bucket-brigade QRAM, we could use an ancilla flag qubit which controls the production of the input photons from the address qubits. If this flag is set to 0, no input photons are generated and nothing enters the QRAM device.

Then we consider the circuit shown in Figure 11. This has m devices to access the same table T . It generates a superposition of m different states, indexing which QRAM device to shuffle our real query into (the query is $|\phi\rangle = \sum_i \alpha_i |i\rangle |0\rangle$), while sending $|\text{nop}\rangle$ states to all the other devices. We then apply all the QRAM devices, un-shuffle the state, and measure the shuffle controls in the Hadamard basis.

Measuring all zeros in the shuffle controls

means the states that went out came back to the registers they started in, enforcing a “sameness” among the queries between different QRAM devices. If the errors are independent, then this suppresses errors, since it’s unlikely for the same error to occur in all QRAM devices at the same time.

[58] proposes compressing this to use only one QRAM device by sequentially controlling swaps, so that only one of m applications of the QRAM applies to the real query state. Since this error correction method only works for uncorrelated errors, it fails for bucket-brigade QRAM. Specifically, if an error occurs at some point in the routing nodes, that error will persist between queries (unless the routing nodes are reset, which creates an $\Omega(N)$ cost as discussed previously). Thus, on average half of all errors will be identical between queries, rendering the error suppression ineffective.

Expanding their idea to a wide, parallel repetition increases hardware overhead by a factor of m , but we imagine $m \ll N$, otherwise the error suppression scheme – which requires $\Omega(m \log N)$ gates for the shuffles alone – will start to approach the cost of a circuit QRAM access.

The overall effect of this is a “heralded” QRAM access, where if we measure all zeros in the control qubits, we know that the infidelity has been suppressed to $1/m$ of the physical error rate (see [58, Chapter 4] for proof), *but* the probability p of measuring all-zero is approximately equal to the original physical infidelity. This means that for an algorithm to have a reasonable probability of success, it must be limited to $O(\frac{1}{1-p})$ QRAM queries. Recalling the applications from Section 3, this forbids database QAA.

Finally, the code switching process raises some issues. Imagine Figure 11 in the surface code. We can attempt to limit the time when a qubit is outside of the surface code to the smallest possible, but even a brief period without error correction could spell disaster for a resource-intensive algorithm.

10.5 Two-Level Quantum Walkers

A recent proposal for ballistic quantum computation imagines “walkers” (e.g., fermions) propagating along a waveguide [6, 7]. Each walker is a two-level quantum system, but they also use a dual-rail encoding. A special “roundabout gate”

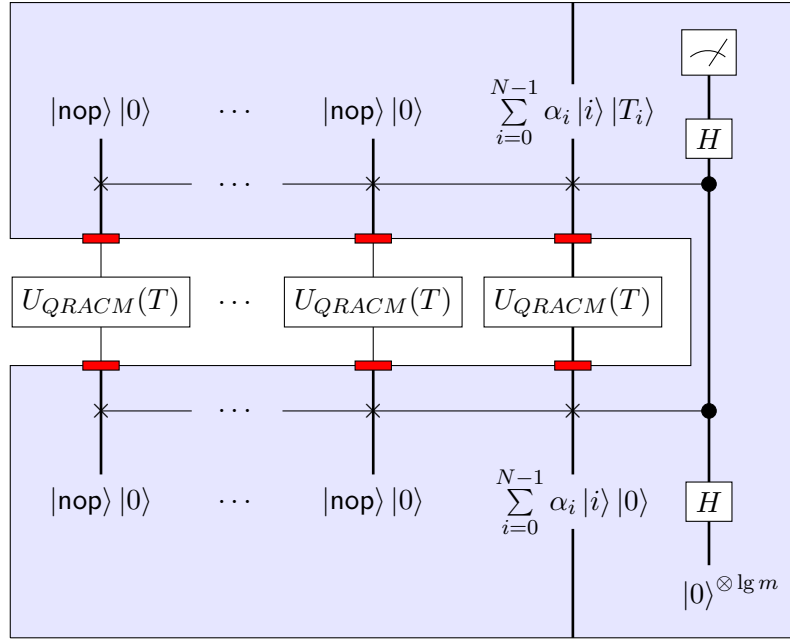


Figure 11: QRACM with heralded error mitigation, adapted from [58]. Shaded regions and thicker wires indicate error corrected qubits and operations, and red rectangles on the border indicate decoding and re-encoding. The controlled swaps indicate that, based on the $\lg m$ control qubits, the last register is swapped to one of the intermediate registers. The states shown at the top are the states if no errors occur, in which case the measurement result is all 0.

can switch between these encodings by rotating $|0\rangle$ states of the walker clockwise, and $|1\rangle$ states counterclockwise. Combined with single qubit rotations on each walker (such as magnetic fields if the walkers are fermions), they construct a universal, ballistic gate set. In [7] they use these gates to construct QRAM. They route each address walker through a binary tree, with CNOT gates between each edge in the tree to perform the routing.

If the underlying ballistic computation were built, then this would work as QRAM; however, such a technology faces all the issues we discuss in Section 6. In fact, their proposal can construct a ballistic version of *any* circuit, which would be a powerful and revolutionary quantum technology.

To function properly, the walkers must transition correctly at the roundabout gates, and [6] computes the probability of this transition in terms of the momentum of the wave packets. If the momentum differs from the target momentum by ϵ , the transition probability scales as $1 - \Omega(\epsilon^2)$. Keeping the momentum of a wavepacket constrained requires a broader wavepacket, and an exponentially precise momentum in their system requires an exponentially wide wavepacket. Thus, their proposed system needs improvement to accommodate exponential-sized circuits.

However, it may be that the error rates of their QRAM access scale like the bucket-brigade, and the momentum precision only needs to be logarithmic in the memory size. In this case the quantum walkers could act as a specialized QRAM module, but more work must be done to clarify the error rates and to avoid the issues we have discussed in the interfaces between the quantum walkers and the rest of a fault-tolerant computation.

11 Discussion

Large scale, high quality QRAM which is cheap to use is unlikely. It would require passively-corrected quantum memory and highly complex, high-fidelity ballistic computation. We argued in Section 6 and Section 7 that these are much stronger assumptions about quantum computing hardware than general large-scale, fault-tolerant quantum computing. Even if those assumptions come true, QRAM may still be difficult: building a ballistic computation is complicated (e.g. Theorem 6.1) and it may not interact well with the rest of a fault-tolerant quantum computation (Theorem 7.1). Indeed, we could find no proposed hardware-QRAM technology that was not implicitly active or required unreasonably low er-

ror rates (Section 9, Section 10).

All of that said, we cannot *prove* that cheap QRAM will not be possible. However, many powerful computing devices (say, a polynomial time SAT-solver) are not provably impossible, but we do not assume they will be built. Our claim is that cheap QRAM should be considered infeasible until shown otherwise, instead of the reverse. While classical RAM appears to set a precedent for such devices, we argued in Section 8 that this is an inappropriate comparison: QRAM is more difficult relative to other quantum computations, than classical RAM is difficult relative to classical computation.

Despite these critiques, QRAM will likely be a crucial tool for quantum algorithms. In a circuit model, QRAM has a linear gate cost (regardless of depth), but optimized circuits allow it to help with quantum chemistry and some cryptographic attacks (Section 5). We encourage readers to consider circuit QRAM for their algorithms, and use one of the optimized circuits we referenced.

We leave here a summary of some open questions we raised in this work:

1. With the favourable error scaling of bucket-brigade QRACM, can it be more effective than other methods in a surface code, especially for cryptographic attacks?
2. If we correct only some of the nodes in a bucket-brigade QRAM, can this suppress enough noise for practical purposes?
3. Can Theorem 5.2 and Theorem 7.1 extend to account for measurement feedback, and can the latter extend to data-specific distillation processes?
4. How should we model the errors and the costs of construction and use of large-scale Hamiltonians for ballistic computations?
5. Are there codes for which QRAM is transversal, or can otherwise be applied directly to logical states?
6. Can we fix the scaling issues of the transmon-based bucket-brigade memory of [29]?
7. How much noise can fault-tolerant quantum linear algebra tolerate in the QRAM access, and could bucket-brigade QRAM reach that level?

ACKNOWLEDGEMENTS

We thank John Schanck for many fruitful discussions and providing many of these arguments; Simon C. Benjamin, Balint Koczor, Sam McArdle, Shouvanik Chakrabarti, Dylan Herman, Yue Sun, Patrick Rebertrost, Matteo Votto, and an anonymous reviewer for helpful comments; and Romy Minko, Ryan Mann, Oliver Brown, and Christian Majenz for valiant attempts to generalize Theorem C.2. S.J. was funded by the University of Oxford Clarendon fund, and acknowledges the support of the Natural Sciences and Engineering Research Council (NSERC), funding reference number RGPIN-2024-03996. A.G.R. was funded by a JPMorgan Chase Global Technology Applied Research PhD Fellowship and the SEEQA project (EP/Y004655/1).

DISCLAIMER

This research was funded in part by JPMorgan Chase & Co. Any views or opinions expressed herein are solely those of the authors listed, and may differ from the views and opinions expressed by JPMorgan Chase & Co. or its affiliates. This material is not a product of the Research Department of J.P. Morgan Securities LLC. This material should not be construed as an individual recommendation for any particular client and is not intended as a recommendation of particular securities, financial instruments or strategies for a particular client. This material does not constitute a solicitation or offer in any jurisdiction.

References

- [1] Scott Aaronson. Read the fine print. *Nature Physics*, 11(4):291–293, April 2015. DOI: [10.1038/nphys3272](https://doi.org/10.1038/nphys3272).
- [2] Martin R. Albrecht and Yixin Shen. Quantum augmented dual attack. *Cryptology ePrint Archive*, Paper 2022/656, 2022.
- [3] Martin R. Albrecht, Vlad Gheorghiu, Eamonn W. Postlethwaite, and John M. Schanck. Estimating quantum speedups for lattice sieves. In Shiho Moriai and Huaxiong Wang, editors, *Advances in Cryptology – ASIACRYPT 2020*, pages 583–613, Cham, 2020. Springer International Publishing. ISBN 978-3-030-64834-

3. DOI: https://doi.org/10.1007/978-3-030-64834-3_20.
- [4] AMD. AMD instinct™ MI250X accelerator, 2022. URL <https://www.amd.com/en/products/server-accelerators/instinct-mi250x>.
- [5] Srinivasan Arunachalam, Vlad Gheorghiu, Tomas Jochym-O'Connor, Michele Mosca, and Priyaa Varshinee Srinivasan. On the robustness of bucket brigade quantum RAM. *New Journal of Physics*, 17(12):123010, December 2015. DOI: [10.1088/1367-2630/17/12/123010](https://doi.org/10.1088/1367-2630/17/12/123010).
- [6] Ryo Asaka, Kazumitsu Sakai, and Ryoko Yahagi. Quantum random access memory via quantum walk. *Quantum Science and Technology*, 6(3):035004, 2021. DOI: <https://doi.org/10.1088/2058-9565/abf484>.
- [7] Ryo Asaka, Kazumitsu Sakai, and Ryoko Yahagi. Two-level quantum walkers on directed graphs. ii. application to quantum random access memory. *Phys. Rev. A*, 107:022416, Feb 2023. DOI: [10.1103/PhysRevA.107.022416](https://doi.org/10.1103/PhysRevA.107.022416).
- [8] Ryan Babbush, Craig Gidney, Dominic W. Berry, Nathan Wiebe, Jarrod McClean, Alexandru Paler, Austin Fowler, and Hartmut Neven. Encoding electronic spectra in quantum circuits with linear t complexity. *Phys. Rev. X*, 8:041015, Oct 2018. DOI: [10.1103/PhysRevX.8.041015](https://doi.org/10.1103/PhysRevX.8.041015).
- [9] Ryan Babbush, Jarrod R. McClean, Michael Newman, Craig Gidney, Sergio Boixo, and Hartmut Neven. Focus beyond quadratic speedups for error-corrected quantum advantage. *PRX Quantum*, 2:010103, Mar 2021. DOI: <https://doi.org/10.1103/PRXQuantum.2.010103>.
- [10] Ryan Babbush, Jarrod R. McClean, Michael Newman, Craig Gidney, Sergio Boixo, and Hartmut Neven. Focus beyond quadratic speedups for error-corrected quantum advantage. *PRX Quantum*, 2(1):010103, 2021.
- [11] Ainesh Bakshi and Ewin Tang. *An Improved Classical Singular Value Transformation for Quantum Machine Learning*, pages 2398–2453. DOI: [10.1137/1.9781611977912.86](https://doi.org/10.1137/1.9781611977912.86).
- [12] Gustavo Banegas, Daniel J. Bernstein, Iggy Van Hoof, and Tanja Lange. Concrete quantum cryptanalysis of binary elliptic curves. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, pages 451–472, December 2020. DOI: [10.46586/tches.v2021.i1.451-472](https://doi.org/10.46586/tches.v2021.i1.451-472).
- [13] Robert Beals, Stephen Brierley, Oliver Gray, Aram W. Harrow, Samuel Kutin, Noah Linden, Dan Shepherd, and Mark Stather. Efficient distributed quantum computing. *Proceedings of the Royal Society A: Mathematical, Physical and Engineering Sciences*, 469(2153):20120686, May 2013. DOI: [10.1098/rspa.2012.0686](https://doi.org/10.1098/rspa.2012.0686).
- [14] Akrem Benatia, Weixing Ji, Yizhuo Wang, and Feng Shi. Sparse matrix partitioning for optimizing SpMV on CPU-GPU heterogeneous platforms. *The International Journal of High Performance Computing Applications*, 34(1):66–80, November 2019. DOI: [10.1177/1094342019886628](https://doi.org/10.1177/1094342019886628).
- [15] D. J. Bernstein. Circuits for integer factorization: a proposal, 2001. <https://cr.yp.to/papers.html#nfscircuit>.
- [16] D. J. Bernstein. Cost analysis of hash collisions: Will quantum computers make shares obsolete? *Workshop Record of SHARCS'09: Special-purpose Hardware for Attacking Cryptographic Systems*, 2009.
- [17] Daniel J. Bernstein and Tanja Lange. Non-uniform cracks in the concrete: The power of free precomputation. In Kazue Sako and Palash Sarkar, editors, *Advances in Cryptology - ASIACRYPT 2013*, pages 321–340, Berlin, Heidelberg, 2013. Springer Berlin Heidelberg. ISBN 978-3-642-42045-0. DOI: https://doi.org/10.1007/978-3-642-42045-0_17.
- [18] Daniel J. Bernstein, Stacey Jeffery, Tanja Lange, and Alexander Meurer. Quantum algorithms for the subset-sum problem. In Philippe Gaborit, editor, *Post-Quantum Cryptography*, pages 16–33, Berlin, Heidelberg, 2013. Springer Berlin Heidelberg. ISBN 978-3-642-38616-9. DOI: https://doi.org/10.1007/978-3-642-38616-9_2.
- [19] Dominic W. Berry, Craig Gidney, Mario Motta, Jarrod R. McClean, and Ryan Babbush. Qubitization of Arbitrary Basis Quantum Chemistry Leveraging Sparsity

- and Low Rank Factorization. *Quantum*, 3:208, December 2019. ISSN 2521-327X. DOI: [10.22331/q-2019-12-02-208](https://doi.org/10.22331/q-2019-12-02-208).
- [20] Mihir K. Bhaskar, Stuart Hadfield, Anargyros Papageorgiou, and Iasonas Petras. Quantum algorithms and circuits for scientific computing. *Quantum Info. Comput.*, 16(3–4):197–236, March 2016. ISSN 1533-7146. DOI: <https://doi.org/10.26421/qic16.3-4-2>.
- [21] Jacob Biamonte, Peter Wittek, Nicola Pancotti, Patrick Rebentrost, Nathan Wiebe, and Seth Lloyd. Quantum machine learning. *Nature*, 549(7671):195–202, 2017. DOI: <https://doi.org/10.1038/nature23474>.
- [22] G. Bilardi and F.P. Preparata. Horizons of parallel computation. *Journal of Parallel and Distributed Computing*, 27(2):172–182, 1995. ISSN 0743-7315. DOI: <https://doi.org/10.1006/jpdc.1995.1080>.
- [23] Xavier Bonnetain and André Schrottenloher. Quantum security analysis of CSIDH. In *Advances in Cryptology – EUROCRYPT 2020*, pages 493–522. Springer International Publishing, 2020. DOI: [10.1007/978-3-030-45724-2_17](https://doi.org/10.1007/978-3-030-45724-2_17).
- [24] Xavier Bonnetain, André Chailloux, André Schrottenloher, and Yixin Shen. Finding many collisions via reusable quantum walks: Application to lattice sieving. In *Advances in Cryptology – EUROCRYPT 2023: 42nd Annual International Conference on the Theory and Applications of Cryptographic Techniques, Lyon, France, April 23-27, 2023, Proceedings, Part V*, page 221–251, Berlin, Heidelberg, 2023. Springer-Verlag. ISBN 978-3-031-30588-7. DOI: https://doi.org/10.1007/978-3-031-30589-4_8.
- [25] Gilles Brassard, Peter Høyer, and Alain Tapp. Quantum cryptanalysis of hash and claw-free functions. *ACM SIGACT News*, 28(2):14–19, June 1997. DOI: [10.1145/261342.261346](https://doi.org/10.1145/261342.261346).
- [26] Gilles Brassard, Peter Høyer, Michele Mosca, and Alain Tapp. Quantum amplitude amplification and estimation. *Quantum Computation and Information*, page 53–74, 2002. ISSN 0271-4132. DOI: [10.1090/conm/305/05215](https://doi.org/10.1090/conm/305/05215). URL <http://dx.doi.org/10.1090/conm/305/05215>.
- [27] Benjamin J. Brown, Daniel Loss, Jianis K. Pachos, Chris N. Self, and James R. Wootton. Quantum memories at finite temperature. *Reviews of Modern Physics*, 88(4), November 2016. DOI: [10.1103/revmodphys.88.045005](https://doi.org/10.1103/revmodphys.88.045005).
- [28] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel Ziegler, Jeffrey Wu, Clemens Winter, Chris Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language models are few-shot learners. In H. Larochelle, M. Ranzato, R. Hadsell, M.F. Balcan, and H. Lin, editors, *Advances in Neural Information Processing Systems*, volume 33, pages 1877–1901. Curran Associates, Inc., 2020.
- [29] Arnau Sala Cadellans. A transmon based quantum switch for a quantum random access memory. Master’s thesis, Leiden University Faculty of Science, Leiden, Netherlands, 2015.
- [30] Yu Cai, Saugata Ghose, Erich F. Haratsch, Yixin Luo, and Onur Mutlu. Error characterization, mitigation, and recovery in flash-memory-based solid-state drives. *Proceedings of the IEEE*, 105(9):1666–1704, 2017. DOI: [10.1109/JPROC.2017.2713127](https://doi.org/10.1109/JPROC.2017.2713127).
- [31] Aaron Carroll and Gernot Heiser. An analysis of power consumption in a smartphone. In *Proceedings of the 2010 USENIX Conference on USENIX Annual Technical Conference*, USENIXATC’10, page 21, USA, 2010. USENIX Association.
- [32] Tore Vincent Carstens and Dirk Oliver Theis. Note on (active-)qram-style data access as a quantum circuit, 2018. URL <https://arxiv.org/abs/1810.10759>.
- [33] André Chailloux and Johanna Loyer. Lattice sieving via quantum random walks. In Mehdi Tibouchi and Huaxiong Wang, editors, *Advances in Cryptology –*

- ASIACRYPT 2021*, pages 63–91, Cham, 2021. Springer International Publishing. ISBN 978-3-030-92068-5. DOI: https://doi.org/10.1007/978-3-030-92068-5_3.
- [34] Jorge Chávez-Saab, Jesús-Javier Chi-Domínguez, Samuel Jaques, and Francisco Rodríguez-Henríquez. The SQALE of CSIDH: sublinear vélu quantum-resistant isogeny action with low exponents. *Journal of Cryptographic Engineering*, August 2021. DOI: [10.1007/s13389-021-00271-w](https://doi.org/10.1007/s13389-021-00271-w).
- [35] Kevin C. Chen, Wenhan Dai, Carlos Errando-Herranz, Seth Lloyd, and Dirk Englund. Heralded quantum random access memory in a scalable photonic integrated circuit platform. In *Conference on Lasers and Electro-Optics*. Optica Publishing Group, 2021. DOI: [10.1364/cleo-qels.2021.fth1p.7](https://doi.org/10.1364/cleo-qels.2021.fth1p.7).
- [36] Nadiia Chepurko, Kenneth Clarkson, Lior Horesh, Honghao Lin, and David Woodruff. Quantum-inspired algorithms from randomized numerical linear algebra. In *International Conference on Machine Learning*, pages 3879–3900. PMLR, 2022.
- [37] Nai-Hui Chia, András Pal Gilyén, Tongyang Li, Han-Hsuan Lin, Ewin Tang, and Chunhao Wang. Sampling-based sublinear low-rank matrix arithmetic framework for dequantizing quantum machine learning. *Journal of the ACM*, 69(5):1–72, 2022. DOI: <https://doi.org/10.1145/3549524>.
- [38] Charles Q. Choi. The beating heart of the world’s first exascale supercomputer. *IEEE Spectrum*, 2022. <https://spectrum.ieee.org/frontier-exascale-supercomputer>.
- [39] Carlo Ciliberto, Mark Herbster, Alessandro Davide Ialongo, Massimiliano Pontil, Andrea Rocchetto, Simone Severini, and Leonard Wossnig. Quantum machine learning: a classical perspective. *Proceedings of the Royal Society A: Mathematical, Physical and Engineering Sciences*, 474(2209):20170551, January 2018. DOI: [10.1098/rspa.2017.0551](https://doi.org/10.1098/rspa.2017.0551).
- [40] B. David Clader, Alexander M. Dalzell, Nikitas Stamatopoulos, Grant Salton, Mario Berta, and William J. Zeng. Quantum resources required to block-encode a matrix of classical data, 2022.
- [41] Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. Llm.int8(): 8-bit matrix multiplication for transformers at scale. In *Proceedings of the 36th International Conference on Neural Information Processing Systems, NIPS ’22*, Red Hook, NY, USA, 2022. Curran Associates Inc. ISBN 9781713871088.
- [42] Laurent El Ghaoui. Inversion error, condition number, and approximate inverses of uncertain matrices. *Linear algebra and its applications*, 343:171–193, 2002. DOI: [https://doi.org/10.1016/S0024-3795\(01\)00273-7](https://doi.org/10.1016/S0024-3795(01)00273-7).
- [43] Richard P. Feynman. Quantum mechanical computers. *Optics News*, 11(2):11–20, Feb 1985. DOI: [10.1364/ON.11.2.000011](https://doi.org/10.1364/ON.11.2.000011).
- [44] Elias Frantar, Saleh Ashkboos, Torsten Hoefer, and Dan Alistarh. Gptq: Accurate post-training quantization for generative pre-trained transformers. *arXiv preprint arXiv:2210.17323*, 2022.
- [45] Craig Gidney. Windowed quantum arithmetic. *arXiv:1905.07682*, 2019.
- [46] Craig Gidney. Quantum dictionaries without gram. *arXiv:2204.13835*, 2022.
- [47] Craig Gidney and Martin Ekerå. How to factor 2048 bit RSA integers in 8 hours using 20 million noisy qubits. *Quantum*, 5:433, April 2021. ISSN 2521-327X. DOI: [10.22331/q-2021-04-15-433](https://doi.org/10.22331/q-2021-04-15-433).
- [48] András Gilyén, Seth Lloyd, and Ewin Tang. Quantum-inspired low-rank stochastic regression with logarithmic dependence on the dimension. *arXiv preprint arXiv:1811.04909*, 2018.
- [49] András Gilyén, Yuan Su, Guang Hao Low, and Nathan Wiebe. Quantum singular value transformation and beyond: exponential improvements for quantum matrix arithmetics. In *Proceedings of the 51st Annual ACM SIGACT Symposium on Theory of Computing*, pages 193–204, 2019. DOI: <https://doi.org/10.1145/3313276.3316366>.
- [50] R. Gioiosa. Resilience for extreme scale computing. In *Rugged Embedded Systems*, pages 123–148. Elsevier, 2017. DOI: [10.1016/b978-0-12-802459-1.00005-1](https://doi.org/10.1016/b978-0-12-802459-1.00005-1).
- [51] Vittorio Giovannetti, Seth Lloyd, and

- Lorenzo Maccone. Architectures for a quantum random access memory. *Phys. Rev. A*, 78:052310, Nov 2008. DOI: [10.1103/PhysRevA.78.052310](https://doi.org/10.1103/PhysRevA.78.052310).
- [52] Vittorio Giovannetti, Seth Lloyd, and Lorenzo Maccone. Quantum random access memory. *Phys. Rev. Lett.*, 100:160501, Apr 2008. DOI: [10.1103/PhysRevLett.100.160501](https://doi.org/10.1103/PhysRevLett.100.160501).
- [53] John T. Godfrey. Binary digital computer, U.S. Patent 3390471A, Jul. 1968.
- [54] Google. Cloud TPU documentation: System architecture, 2022. <https://cloud.google.com/tpu/docs/system-architecture-tpu-vm>.
- [55] Ananth Grama, Anshul Gupta, George Karypis, and Vipin Kumar. *Introduction to Parallel Computing*. Addison Wesley, 2003. ISBN 0-201-64865-2.
- [56] J. Alex Halderman, Seth D. Schoen, Nadia Heninger, William Clarkson, William Paul, Joseph A. Calandrino, Ariel J. Feldman, Jacob Appelbaum, and Edward W. Felten. Lest we remember: Cold-boot attacks on encryption keys. *Commun. ACM*, 52(5):91–98, may 2009. ISSN 0001-0782. DOI: [10.1145/1506409.1506429](https://doi.org/10.1145/1506409.1506429).
- [57] Thomas Häner, Samuel Jaques, Michael Naehrig, Martin Roetteler, and Mathias Soeken. Improved quantum circuits for elliptic curve discrete logarithms. In *Post-Quantum Cryptography*, pages 425–444. Springer International Publishing, 2020. DOI: [10.1007/978-3-030-44223-1_23](https://doi.org/10.1007/978-3-030-44223-1_23).
- [58] Connor T. Hann. *Practicality of Quantum Random Access Memory*. PhD thesis, Yale Graduate School of Arts and Sciences Dissertations, 2021.
- [59] Connor T. Hann, Gideon Lee, S.M. Girvin, and Liang Jiang. Resilience of quantum random access memory to generic noise. *PRX Quantum*, 2(2), April 2021. DOI: [10.1103/prxquantum.2.020311](https://doi.org/10.1103/prxquantum.2.020311).
- [60] Aram W. Harrow, Avinatan Hassidim, and Seth Lloyd. Quantum algorithm for linear systems of equations. *Phys. Rev. Lett.*, 103:150502, Oct 2009. DOI: [10.1103/PhysRevLett.103.150502](https://doi.org/10.1103/PhysRevLett.103.150502).
- [61] Max Heiser. Improved quantum hypercone locality sensitive filtering in lattice sieving. Cryptology ePrint Archive, Paper 2021/1295, 2021. <https://eprint.iacr.org/2021/1295>.
- [62] Fang-Yu Hong, Yang Xiang, Zhi-Yan Zhu, Li-zhen Jiang, and Liang-neng Wu. Robust quantum random access memory. *Phys. Rev. A*, 86:010306, Jul 2012. DOI: [10.1103/PhysRevA.86.010306](https://doi.org/10.1103/PhysRevA.86.010306).
- [63] Samuel Jaques and John M. Schanck. Quantum cryptanalysis in the RAM model: Claw-finding attacks on SIKE. In Alexandra Boldyreva and Daniele Micciancio, editors, *Advances in Cryptology - CRYPTO 2019 - 39th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 18-22, 2019, Proceedings, Part I*, volume 11692 of *Lecture Notes in Computer Science*, pages 32–61. Springer, 2019. DOI: [10.1007/978-3-030-26948-7_2](https://doi.org/10.1007/978-3-030-26948-7_2). URL https://doi.org/10.1007/978-3-030-26948-7_2.
- [64] N. Jiang, Y.-F. Pu, W. Chang, C. Li, S. Zhang, and L.-M. Duan. Experimental realization of 105-qubit random access quantum memory. *npj Quantum Information*, 5(1), April 2019. DOI: [10.1038/s41534-019-0144-0](https://doi.org/10.1038/s41534-019-0144-0).
- [65] Jung Jun Park, Kyunghyun Baek, M S Kim, Hyunchul Nha, Jaewan Kim, and Jeongho Bang. T-depth-optimized quantum search with quantum data-access machine. *Quantum Science and Technology*, 9(1):015011, nov 2023. DOI: [10.1088/2058-9565/ad04e5](https://doi.org/10.1088/2058-9565/ad04e5). URL <https://doi.org/10.1088/2058-9565/ad04e5>.
- [66] Ghazal Kachigar and Jean-Pierre Tillich. Quantum information set decoding algorithms, 2017. URL [arXiv:1703.00263](https://arxiv.org/abs/1703.00263).
- [67] Dhiraj Kalamkar, Dheevatsa Mudigere, Naveen Mellempudi, Dipankar Das, Kunal Banerjee, Sasikanth Avancha, Dharma Teja Vooturi, Nataraj Jammalamadaka, Jianyu Huang, Hector Yuen, et al. A study of bfloat16 for deep learning training. *arXiv preprint arXiv:1905.12322*, 2019.
- [68] Ravindran Kannan and Santosh Vempala. Randomized algorithms in numerical linear algebra. *Acta Numerica*, 26:95–135, 2017. DOI: [10.1017/S0962492917000058](https://doi.org/10.1017/S0962492917000058).
- [69] Iordanis Kerenidis and Anupam Prakash.

- Quantum recommendation systems. *arXiv preprint arXiv:1603.08675*, 2016.
- [70] Elena Kirshanova. Improved quantum information set decoding. In Tanja Lange and Rainer Steinwandt, editors, *Post-Quantum Cryptography*, pages 507–527, Cham, 2018. Springer International Publishing. ISBN 978-3-319-79063-3. DOI: https://doi.org/10.1007/978-3-319-79063-3_24.
 - [71] A. Yu. Kitaev, A. Shen, and M. N. Viazovkin. *Classical and Quantum Computation*. American Mathematical Society, USA, 2002. ISBN 082182161X. DOI: <http://dx.doi.org/10.1090/gsm/047/31>.
 - [72] G. Kuperberg. Another subexponential-time quantum algorithm for the dihedral hidden subgroup problem. In *TQC 2013*, LIPIcs 22, pages 20–34, 2013. DOI: [10.4230/LIPIcs.TQC.2013.20](https://doi.org/10.4230/LIPIcs.TQC.2013.20). URL <https://doi.org/10.4230/LIPIcs.TQC.2013.20>.
 - [73] Thijs Laarhoven. Sieving for closest lattice vectors (with preprocessing). In Roberto Avanzi and Howard Heys, editors, *Selected Areas in Cryptography – SAC 2016*, pages 523–542, Cham, 2017. Springer International Publishing. ISBN 978-3-319-69453-5. DOI: https://doi.org/10.1007/978-3-319-69453-5_28.
 - [74] Thijs Laarhoven, Michele Mosca, and Joop van de Pol. Finding shortest lattice vectors faster using quantum search. *Designs, Codes and Cryptography*, 77(2-3):375–400, April 2015. DOI: [10.1007/s10623-015-0067-5](https://doi.org/10.1007/s10623-015-0067-5).
 - [75] C. Lecaplain, C. Javerzac-Galy, M. L. Gorodetsky, and T. J. Kippenberg. Mid-infrared ultra-high-q resonators based on fluoride crystalline materials. *Nature Communications*, 7(1), November 2016. DOI: [10.1038/ncomms13383](https://doi.org/10.1038/ncomms13383).
 - [76] Lin Lin. Lecture notes on quantum algorithms for scientific computation. *arXiv preprint arXiv:2201.08309*, 2022.
 - [77] Junyu Liu, Minzhao Liu, Jin-Peng Liu, Ziyu Ye, Yunfei Wang, Yuri Alexeev, Jens Eisert, and Liang Jiang. Towards provably efficient quantum algorithms for large-scale machine-learning models. *Nature Commu-*
 - nications*, 15(1), January 2024. ISSN 2041-1723. DOI: [10.1038/s41467-023-43957-x](https://doi.org/10.1038/s41467-023-43957-x).
 - [78] Guang Hao Low and Isaac L Chuang. Hamiltonian simulation by qubitization. *Quantum*, 3:163, 2019. DOI: <https://doi.org/10.22331/q-2019-07-12-163>.
 - [79] Guang Hao Low, Vadym Kliuchnikov, and Luke Schaeffer. Trading T gates for dirty qubits in state preparation and unitary synthesis, June 2024. ISSN 2521-327X. URL <https://doi.org/10.22331/q-2024-06-17-1375>.
 - [80] John M Martyn, Zane M Rossi, Andrew K Tan, and Isaac L Chuang. Grand unification of quantum algorithms. *PRX Quantum*, 2(4):040203, 2021. DOI: <https://doi.org/10.1103/PRXQuantum.2.040203>.
 - [81] Olivia Di Matteo, Vlad Gheorghiu, and Michele Mosca. Fault-tolerant resource estimation of quantum random-access memories. *IEEE Transactions on Quantum Engineering*, 1:1–13, 2020. DOI: [10.1109/TQE.2020.2965803](https://doi.org/10.1109/TQE.2020.2965803).
 - [82] Vidyabhushan Mohan. *Modelling the physical characteristics of NAND flash memory*. PhD thesis, School of Engineering and Applied Science, University of Virginia, 2010.
 - [83] E. S. Moiseev and S. A. Moiseev. Time-bin quantum RAM. *Journal of Modern Optics*, 63(20):2081–2092, May 2016. DOI: [10.1080/09500340.2016.1182222](https://doi.org/10.1080/09500340.2016.1182222).
 - [84] Dustin Moody. Status report on the third round of the NIST post-quantum cryptography standardization process. Technical report, National Institute of Standards and Technology, 2022.
 - [85] Michele Mosca and Marco Piani. Quantum threat timeline report 2020. Technical report, Global Risk Institute, 2021.
 - [86] Michael Newman and Yaoyun Shi. Limitations on transversal computation through quantum homomorphic encryption. *Quantum Info. Comput.*, 18(11–12):927–948, sep 2018. ISSN 1533-7146. DOI: <https://doi.org/10.26421/qic18.11-12-3>.
 - [87] Murphy Yuezhen Niu, Alexander Zlokapa, Michael Broughton, Sergio Boixo, Masoud Mohseni, Vadim Smelyanskiy, and Hartmut Neven. Entangling quantum genera-

- tive adversarial networks. *Phys. Rev. Lett.*, 128:220505, Jun 2022. DOI: [10.1103/PhysRevLett.128.220505](https://doi.org/10.1103/PhysRevLett.128.220505).
- [88] James O’Sullivan, Oscar W. Kennedy, Kamanasish Debnath, Joseph Alexander, Christoph W. Zollitsch, Mantas Šimėnas, Akel Hashim, Christopher N. Thomas, Stafford Withington, Irfan Siddiqi, Klaus Mølmer, and John J. L. Morton. Random-access quantum memory using chirped pulse phase encoding. *Phys. Rev. X*, 12: 041014, Nov 2022. DOI: [10.1103/PhysRevX.12.041014](https://doi.org/10.1103/PhysRevX.12.041014).
- [89] Alexandru Paler, Oumarou Oumarou, and Robert Basmadjian. Parallelizing the queries in a bucket-brigade quantum random access memory. *Phys. Rev. A*, 102: 032608, Sep 2020. DOI: [10.1103/PhysRevA.102.032608](https://doi.org/10.1103/PhysRevA.102.032608).
- [90] Daniel K. Park, Francesco Petruccione, and June-Koo Kevin Rhee. Circuit-based quantum random access memory for classical data. *Scientific Reports*, 9(1), March 2019. DOI: [10.1038/s41598-019-40439-3](https://doi.org/10.1038/s41598-019-40439-3).
- [91] Fernando Pastawski and Beni Yoshida. Fault-tolerant logical gates in quantum error-correcting codes. *Phys. Rev. A*, 91: 012305, Jan 2015. DOI: [10.1103/PhysRevA.91.012305](https://doi.org/10.1103/PhysRevA.91.012305).
- [92] Chris Peikert. He gives c-sieves on the CSIDH. In *Advances in Cryptology – EUROCRYPT 2020*, pages 463–492. Springer International Publishing, 2020. DOI: [10.1007/978-3-030-45724-2_16](https://doi.org/10.1007/978-3-030-45724-2_16).
- [93] Koustubh Phalak, Junde Li, and Swaroop Ghosh. Approximate quantum random access memory architectures, 2022. URL <https://arxiv.org/abs/2210.14804>.
- [94] Koustubh Phalak, Avimita Chatterjee, and Swaroop Ghosh. Quantum random access memory for dummies. *Sensors*, 23 (17), 2023. ISSN 1424-8220. DOI: [10.3390/s23177462](https://doi.org/10.3390/s23177462). URL <https://www.mdpi.com/1424-8220/23/17/7462>.
- [95] Marco Pistoia, Syed Farhan Ahmad, Akshay Ajagekar, Alexander Buts, Shouvanik Chakrabarti, Dylan Herman, Shaohan Hu, Andrew Jena, Pierre Minssen, Pradeep Niroula, et al. Quantum machine learning for finance ICCAD special session paper. In *2021 IEEE/ACM International Conference On Computer Aided Design (ICCAD)*, pages 1–9. IEEE, 2021. DOI: <https://doi.org/10.1109/ICCAD51958.2021.9643469>.
- [96] Arthur G Rattew and Bálint Koczor. Preparing arbitrary continuous functions in quantum registers with logarithmic complexity. *arXiv preprint arXiv:2205.00519*, 2022.
- [97] Patrick Rebentrost, Masoud Mohseni, and Seth Lloyd. Quantum support vector machine for big data classification. *Physical review letters*, 113(13):130503, 2014. DOI: <https://doi.org/10.1103/PhysRevLett.113.130503>.
- [98] Oded Regev and Liron Schiff. Impossibility of a quantum speed-up with a faulty oracle. In Luca Aceto, Ivan Damgård, Leslie Ann Goldberg, Magnús M. Halldórsson, Anna Ingólfssdóttir, and Igor Walukiewicz, editors, *Automata, Languages and Programming*, pages 773–781, Berlin, Heidelberg, 2008. Springer Berlin Heidelberg. ISBN 978-3-540-70575-8. DOI: https://doi.org/10.1007/978-3-540-70575-8_63.
- [99] Chitwan Saharia, William Chan, Saurabh Saxena, Lala Lit, Jay Whang, Emily Denton, Seyed Kamyar Seyed Ghasemipour, Burcu Karagol Ayan, S. Sara Mahdavi, Raphael Gontijo-Lopes, Tim Salimans, Jonathan Ho, David J Fleet, and Mohammad Norouzi. Photorealistic text-to-image diffusion models with deep language understanding. In *Proceedings of the 36th International Conference on Neural Information Processing Systems*, NIPS ’22, Red Hook, NY, USA, 2022. Curran Associates Inc. ISBN 9781713871088.
- [100] John M. Schanck. via Twitter, Jun 2021. <https://web.archive.org/web/20220710183049/https://twitter.com/susurrusus/status/1532131390338215937>.
- [101] Bianca Schroeder, Eduardo Pinheiro, and Wolf-Dietrich Weber. Dram errors in the wild: A large-scale field study. *Commun. ACM*, 54(2):100–107, feb 2011. ISSN 0001-0782. DOI: [10.1145/1897816.1897844](https://doi.org/10.1145/1897816.1897844).
- [102] Claude. E. Shannon. The synthesis of two-terminal switching circuits. *The Bell System Technical Journal*, 28(1):59–98, 1949. DOI: [10.1002/j.1538-7305.1949.tb03624.x](https://doi.org/10.1002/j.1538-7305.1949.tb03624.x).

- [103] Damien Steiger. Racing in parallel: Quantum versus classical, 2016.
- [104] Xiaoming Sun, Guojing Tian, Shuai Yang, Pei Yuan, and Shengyu Zhang. Asymptotically optimal circuit depth for quantum state preparation and general unitary synthesis. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, pages 1–1, 2023. DOI: [10.1109/TCAD.2023.3244885](https://doi.org/10.1109/TCAD.2023.3244885).
- [105] Richard S Sutton and Andrew G Barto. *Reinforcement learning: An introduction*. MIT press, 2018.
- [106] Ewin Tang. A quantum-inspired classical algorithm for recommendation systems. In *Proceedings of the 51st annual ACM SIGACT symposium on theory of computing*, pages 217–228, 2019. DOI: <https://doi.org/10.1145/3313276.3316310>.
- [107] Seiichiro Tani. An improved claw finding algorithm using quantum walk. In *Mathematical Foundations of Computer Science 2007*, pages 536–547. Springer Berlin Heidelberg, 2007. DOI: [10.1007/978-3-540-74456-6_48](https://doi.org/10.1007/978-3-540-74456-6_48).
- [108] Tezzaron Semiconductor, January 2004.
- [109] W. Tittel, M. Afzelius, T. Chanelière, R.L. Cone, S. Kröll, S.A. Moiseev, and M. Sellars. Photon-echo quantum memory in solid state systems. *Laser & Photonics Reviews*, 4(2):244–267, 2010. DOI: <https://doi.org/10.1002/lpor.200810056>.
- [110] Vlatko Vedral, Adriano Barenco, and Artur Ekert. Quantum networks for elementary arithmetic operations. *Physical Review A*, 54(1):147, 1996. DOI: <https://doi.org/10.1103/PhysRevA.54.147>.
- [111] Zhiling Wang, Zenghui Bao, Yan Li, Yukai Wu, Weizhou Cai, Weiting Wang, Xiyue Han, Jiahui Wang, Yipu Song, Luyan Sun, Hongyi Zhang, and Luming Duan. An ultra-high gain single-photon transistor in the microwave regime. *Nature Communications*, 13(1), October 2022. DOI: [10.1038/s41467-022-33921-6](https://doi.org/10.1038/s41467-022-33921-6).
- [112] Pei Yuan and Shengyu Zhang. Optimal (controlled) quantum state preparation and improved unitary synthesis by quantum circuits with any number of ancillary qubits. *Quantum*, 7:956, March 2023. ISSN 2521-327X. DOI: [10.22331/q-2023-03-20-956](https://doi.org/10.22331/q-2023-03-20-956). URL <https://doi.org/10.22331/q-2023-03-20-956>.
- [113] X. X. Yuan, J.-J. Ma, P.-Y. Hou, X.-Y. Chang, C. Zu, and L.-M. Duan. Experimental demonstration of a quantum router. *Scientific Reports*, 5(1), July 2015. DOI: [10.1038/srep12452](https://doi.org/10.1038/srep12452).
- [114] Xiao-Ming Zhang, Tongyang Li, and Xiao Yuan. Quantum state preparation with optimal circuit depth: Implementations and applications. *Phys. Rev. Lett.*, 129:230504, Nov 2022. DOI: <https://doi.org/10.1103/PhysRevLett.129.230504>.
- [115] Xiao-Ming Zhang, Tongyang Li, and Xiao Yuan. Quantum state preparation with optimal circuit depth: Implementations and applications. *Physical Review Letters*, 129(23):230504, 2022.
- [116] Darko Zivanovic, Pouya Esmaili Dokht, Sergi Moré, Javier Bartolome, Paul M. Carpenter, Petar Radojković, and Eduard Ayguadé. Dram errors in the field: A statistical approach. In *Proceedings of the International Symposium on Memory Systems, MEMSYS '19*, page 69–84, New York, NY, USA, 2019. Association for Computing Machinery. ISBN 9781450372060. DOI: [10.1145/3357526.3357558](https://doi.org/10.1145/3357526.3357558).

A Linear Algebra Proofs

Lemma A.1 (Optimality of Quantum Eigenvalue Transform). *Any quantum algorithm implementing a degree- k polynomial of a matrix H (with the polynomial defined on the eigenvalues) requires $\Omega(k)$ queries to O_H , in general.*

Proof. We prove this lower-bound by a reduction to unstructured search, in the QSVT framework (as per [49]), from which the bound immediately follows. We note that the result of [49] almost immediately implies this bound, but we present the following derivation in a format to enhance the clarity of our discussion. First we formally define a block-encoding. As per [49], an $n + a$ -qubit unitary matrix U_A is called an (α, a, ϵ) -block encoding of the n -qubit matrix A if $\|A - \alpha(|0\rangle^{\otimes a} \otimes I_n)U_A(|0\rangle^{\otimes a} \otimes I_n)\|_2 \leq \epsilon$.

Given an unstructured search function $f(x) = 0$ for all $x \neq m$, and $f(m) = 1$ (with m be-

ing unique), specified by an O_f , oracle defined as $O_f |x\rangle = (-1)^{f(x) \oplus 1} |x\rangle$ (where we negate the sign for ease of analysis), we can construct a $(1, 9, 0)$ -block-encoding U_A of $A := \frac{1}{\sqrt{N}} |m\rangle\langle +^n|$, where $|+^n\rangle = H^{\otimes n} |0\rangle$. We give a very inefficient block encoding (in terms of the number of ancillas) for ease of exposition. First, we construct a $(1, 4, 0)$ -block-encoding of $H^{\otimes n} |0\rangle\langle 0| H^{\otimes n}$. This can first be done by constructing a $(1, 2, 0)$ -block-encoding of $|0\rangle\langle 0| = \frac{1}{2}(I_n + (2|0\rangle\langle 0| - I_n))$ (which can be done by applying the standard sum of block encoding lemma [49]) of an n qubit identity matrix with the standard $2|0\rangle\langle 0| - I_n$ unitary from Grover search. We can then use the product of block encodings lemma with $H^{\otimes n}$, $|0\rangle\langle 0|$, and $H^{\otimes n}$ again to get a $(1, 4, 0)$ -block encoding of $|+^n\rangle\langle +^n|$. We can then use the product of block encoding with $I_1 \otimes O_f$ and $|+^n\rangle\langle +^n|$ to get a $(1, 5, 0)$ -block encoding of $O_f |+^n\rangle\langle +^n|$. We can then use the sum of block encodings with $|+^n\rangle\langle +^n|$ and $O_f |+^n\rangle\langle +^n|$ to get a $(1, 9, 0)$ -block-encoding of $\frac{1}{2}((O_f + I)|+^n\rangle\langle +^n|) = \frac{1}{\sqrt{N}} |m\rangle\langle +^n|$, as desired.

We can apply the polynomial approximation of the sign function as per [49] to our block encoding unitary U_A , giving a constant probability of success of measuring the marked state. They give the degree of this (odd degree) polynomial as $k = O(\log(1/\epsilon)/\delta)$, where ϵ is an upper-bound on the maximum deviation of the polynomial approximation to the sign function in the $x \in [-2, 2] \setminus (-\delta, \delta)$. If we set $\delta = \frac{1}{\sqrt{N}}$, then the sign function maps our non-zero singular value to 1, while leaving the zero-valued singular values unchanged. As a result, we can measure the ancilla register in the $|0\rangle$ state with $\propto 1 - \epsilon$ probability, learning the result of the unstructured search problem. If it were possible to implement this degree $\tilde{O}(1/\delta)$ degree-polynomial with fewer than $\Omega(1/\delta)$ queries to U_A , we could solve unstructured search in general with fewer than $\Omega(\sqrt{N})$ queries to the unstructured search oracle, violating well-known lower-bounds for unstructured search. As a consequence, any general algorithm implementing a SVT of a degree- k polynomial of some matrix H must use at least $\Omega(k)$ queries to that matrix.

A polynomial implementing the sign function applied to the eigenvalues of a block encoding of $\tilde{A} := \begin{pmatrix} 0 & A \\ A^\dagger & 0 \end{pmatrix}$ would also solve unstructured

search in the same way (noting that the non-zero eigenvalues of $\tilde{A}$ correspond to $\pm \frac{1}{\sqrt{N}}$ and have associated eigenvectors $\frac{1}{\sqrt{2}}(|0\rangle_1 |m\rangle \pm |1\rangle_1 |+^n\rangle)$. Thus, applying $\text{Sign}(\tilde{A})$ leaves the 0 eigenvalues unchanged, and maps the $\pm \frac{1}{\sqrt{N}}$ eigenvalues to ± 1 (within ϵ distance). Consequently, $\text{Sign}(\tilde{A}) \approx \begin{pmatrix} 0 & |m\rangle\langle +^n| \\ |+^n\rangle\langle m| & 0 \end{pmatrix}$. Applying $\text{Sign}(\tilde{A})$ to an initial state $|1\rangle |+^n\rangle$ then gives the solution to the unstructured search problem, and so the same bound holds for Problem 1. $\square$

Lemma A.2. *Given P classical processors sharing a common memory of access time $O(1)$, a d -sparse matrix $A \in \mathbb{C}^{N \times N}$ and a vector $\mathbf{v} \in \mathbb{C}^N$, we can compute $A\mathbf{v}$ with $\tilde{O}(Nd/P)$ time complexity.*

Proof. This is a standard result; see [55]. Assume we represent A as N lists of elements (j, A_{ij}) , where A_{ij} are the non-zero components in row i , and $\mathbf{v}$ as an N -element array. With fewer than N processors, each processor will iterate through a row of A . For an entry (j, A_{ij}) , it will look up the j th element of $\mathbf{v}$, multiply it by A_{ij} and add that to a running total. This produces one element of $A\mathbf{v}$. Each element can be computed independently, so this parallelizes perfectly. With more than N processors, rows will have multiple processors. The processors for row i can divide the entries of A in that row and compute sub-totals. To compute the full sum, they add their sub-totals together in a tree structure; this tree has depth $O(\log(P/N))$. $\square$

Following the techniques of [15], we use sorting to build matrix multiplication.

Lemma A.3. *Given P parallel processors forming a sorting network that can sort in time S , multiplying an N -dimensional vector $\mathbf{v}$ by a d -sparse $N \times N$ matrix A takes time $O(S + Nd/P + \lg(d))$.*

Proof. First the processors create d copies of $\mathbf{v}$. Each processor will store a block of A or $\mathbf{v}$ in local memory, so by treating A as a block matrix, we suppose each processor has one entry A_{ij} of A or $\mathbf{v}_j$ of $\mathbf{v}$. We assume there is an efficiently-computable function f_j for each column j , which injectively maps indices k from 1 to d to the non-zero elements of that column. Each processor creates a tuple $(k, j, 0, A_{f_j(k)j})$ for components of

A , and $(k, j, \mathbf{v}_j)$ for components of $\mathbf{v}$ (each copy of $\mathbf{v}$ has a distinct value of k). They will then sort this data by the second component, then the first. Because each column of A contains at most d non-zero entries, this ensures each processor with a tuple $(k, j, 0, A_{f_j(k)j})$ is adjacent to a tuple $(k, j, \mathbf{v}_j)$, so these two processors can compute $A_{f_j(k)j}\mathbf{v}_j$ and store the result in the tuple for A . The processors can then discard the tuples for $\mathbf{v}$, and change the first component from k to $f_j(k)$. Then they re-sort so that the rows of A are physically close (e.g., in a square of length $\sqrt{d}$ in 2 dimensions). They then add the entries of $A_{ij}\mathbf{v}_j$ for each row. Cascading these together will take at least $\lg(d)$ sequential additions, but it will be asymptotically less than the sorts of the full matrix and vector. This produces the i^{th} element of $A\mathbf{v}$. Another sort can send these to whatever memory location represents the output. $\square$

Lemma A.4 (Dense Matrix-Vector Multiplication with Local Memory). *A set of $O(N^2 \log(N))$ processors arranged in a 3D grid and with nearly local connectivity, leading to a total wire-length of $\tilde{O}(N^2)$, can implement a matrix-vector multiplication with an $N \times N$ matrix A and an N -dimensional vector $\mathbf{v}$ with $O(\log(N))$ complexity.*

Proof. Consider a grid of $N \times N$ processors, each with local memory, and *no* connections to their nearest neighbors. Assume that each element in A is assigned to a corresponding processor in the grid, i.e. the processor in the i^{th} row and j^{th} column stores element A_{ij} . If our initial vector $\mathbf{v}$ is stored in a set of N data cells, using a stack of $\log(N)$ local processors, of dimension $2 \times N, 4 \times N, \dots, N \times N$ we can recursively spread the elements in the vector to construct an $N \times N$ matrix V such that $V_{ij} = \mathbf{v}_j$. The first layer in this stack has 2 wires per layer, each of length $N/2$, for a total wire-length of $O(N^2)$. The final layer in this stack, the one that connects element V_{ij} to element A_{ij} in the main $N \times N$ grid, has $O(N^2)$ wires, each of length $O(1)$ for a total wire length of $O(N^2)$. The middle layers in the stack similarly have a total wire length of $O(N^2)$ (summing progressively more shorter wires). Thus, the main $N \times N$ processor grid can store $A_{ij}\mathbf{v}_j$ with a total of $O(N^2 \log N)$ ancillary local processors with a total wire length of $O(N^2 \log N)$. We can then repeat this process of spreading out the elements of $\mathbf{v}$ in reverse,

adding another stack of $\log N$ processors of size $N \times N, N \times N/2, \dots, N \times 2, N \times 1$. We then add elements in adjacent cells, sending them to the cell in the layer above, building up the sum of products of the appropriate matrix and vector element. This produces the output vector $A\mathbf{v}$, where the i^{th} row stores $\sum_{j=1}^N A_{ij}\mathbf{v}_j$, with a total wire-length of $O(N^2 \log N)$, and with a total of $O(N^2 \log N)$ processors. $\square$

B Proof of Theorem 5.1

We start by defining the set of circuits $\mathcal{C}(W, D, G, g, k)$ to be all circuits on W qubits, of circuit depth D , using G gates from a set of size g with fanin at most k . Here we assume all gates have depth 1 and we count circuits based on the arrangement of gates, not by function. Define a “circuit” as a function that takes as input a pair of (qubit, time step) and returns as output the particular gate applied to that qubit at that time step.

If two circuits have a different effect on quantum states, they must use different gates, and thus will be distinct circuits by this definition. This definition overcounts circuits, since equivalent circuits are counted twice, but we want to upper bound the number of circuits so this is fine.

Proposition B.1. *Under this notation,*

$$|\mathcal{C}(W, D, G, g, k)| \leq |\mathcal{C}(W, D, \min\{kG, DW\}, Wg, 1)|.$$

Proof. Let U be an ℓ -qubit gate in the gate set of $\mathcal{C}(W, D, g, k)$. We can define ℓ distinct single-qubit gates $U_1, \dots, U_\ell$ from U , expanding the total number of gates from g to gk , and then further define W/ℓ multiples of each of these single qubit gates, i.e., $U_1^{(1)}, \dots, U_\ell^{(1)}, \dots, U_1^{(m)}, \dots, U_\ell^{(m)}$ where $m = \lfloor W/\ell \rfloor$. This gives at most W single-qubit gates from U , so the total number of single-qubit gates defined in this way is at most Wg .

We now define an injective function from $\mathcal{C}(W, D, G, g, k)$ to $\mathcal{C}(W, D, \min\{kG, DW\}, Wg, 1)$. For each circuit, at each timestep we take all ℓ -qubit gates $U^{(1)}, \dots, U^{(m)}$ (we know there are fewer than $m = \lfloor W/\ell \rfloor$ gates of fanin ℓ in one timestep, because there aren’t enough qubits for more!) and decompose each into $U_1^{(i)}, \dots, U_\ell^{(i)}$, such that the qubit mapped to the j^{th} input of the

i th gate is given gate $U_j^{(i)}$. This increases the total number of gates by at most a factor of k , and cannot increase the number of gates above DW .

For example, a CNOT would split into two gates, a “target” gate and a “control” gate, and if these are reversed, the circuit is different (by our counting).

To show injectivity, suppose two circuits C_1 and C_2 in $\mathcal{C}(W, D, G, g, k)$ map to the same circuit in $\mathcal{C}(W, D, kG, gW, 1)$. Let U be any gate in C_1 ; suppose it applies to ℓ qubits $q_1, \dots, q_\ell$ in time step t . This means there is some i such that the image circuit has $U_1^{(i)}, \dots, U_\ell^{(i)}$ applied to $q_1, \dots, q_\ell$ in time step t . By construction of our function, this means there must be some gate in C_2 of the same type as G which is also applied to those same qubits, in the same order, in time step t . Repeating for all gates shows that $C_1 = C_2$. $\square$

Proposition B.2. *In the same notation, $|\mathcal{C}(W, D, G, g, 1)| \leq \binom{DW}{G} g^G$.*

Proof. We can straightforwardly count: There are DW possible “slots” for each gate, depending on which qubit and which time step. We choose G slots, and for each one, we select one of the g gates. $\square$

Lemma B.3. $|\mathcal{C}(W, D, G, g, k)| \leq \binom{DW}{\min\{kG, DW\}} (Wg)^G$.

Proof. We combine Proposition B.1 and Proposition B.2 to obtain the result. $\square$

C Proof of No Distillation

There are two components of the proof: First, we argue that with only d inputs in the QRACM, we can always find some indices which are “under-represented”, i.e., very little of the amplitude of the input state is concentrated on those indices.

Lemma C.1. *Let $\mathcal{H}_1, \dots, \mathcal{H}_d$ be any collection of finite-dimensional Hilbert spaces, and let $\mathcal{H}_{QRACM}$ be the space that U_{QRACM} acts on for tables of size N . Let $\rho_1, \dots, \rho_d$ be any collection of states, each in $B(\mathcal{H}_i \otimes \mathcal{H}_{QRACM})$, i.e., part of the state is in the input space for a QRACM gate. Then there exists some set of ℓ indices such*

that for any two tables T and T' differing only in those ℓ indices, if we define

$$\delta_i := \|U_{QRACM}(T)(\rho_i) - U_{QRACM}(T')(\rho_i)\|_1 \quad (16)$$

(i.e., the trace distance, with an implicit identity channel on $\mathcal{H}_i$) then

$$\sum_{i=1}^d \delta_i \leq d \sqrt{\frac{2\ell}{N}}. \quad (17)$$

This is more general than necessary: we only need $\ell = 1$ to prove our main theorem.

Proof. Because we care about the application of a perfect QRACM gate, we will purify each of these states, so we consider some purification of ρ_i :

$$|\psi_i\rangle = \sum_{j=0}^{N-1} \sum_{b \in \{0,1\}} \alpha_{ijb} |\psi_{ijb}\rangle |j\rangle |b\rangle \quad (18)$$

where we have expressed the input space to the QRACM gate in the computational basis. Here $|\psi_{ijb}\rangle$ is in $\mathcal{H}_i \otimes \mathcal{H}_p$, where $\mathcal{H}_p$ is the space necessary to purify ρ_i . We see that $U_{QRACM}(T) = \sum_{j,b} |j\rangle\langle j| \otimes |b \oplus T[j]\rangle\langle b|$, so that

$$\begin{aligned} & U_{QRACM}(T')^\dagger U_{QRACM}(T) \\ &= \sum_{j:T'[j]=T[j]} |j\rangle\langle j| \otimes I_2 + \sum_{j:T'[j] \neq T[j]} |j\rangle\langle j| \otimes X \end{aligned} \quad (19)$$

where X is the Pauli X . This implies that

$$\langle \psi_i | U_{QRACM}(T')^\dagger U_{QRACM}(T) | \psi_i \rangle \quad (20)$$

$$= \sum_{j:T[j]=T'[j], b \in \{0,1\}} |\alpha_{ijb}|^2 + \quad (21)$$

$$\sum_{j:T[j] \neq T'[j]} \overline{\alpha_{ij0}} \alpha_{ij1} \langle \psi_{ij0} | \psi_{ij1} \rangle + \overline{\alpha_{ij1}} \alpha_{ij0} \langle \psi_{ij1} | \psi_{ij0} \rangle \quad (22)$$

$$\geq 1 - \left(\sum_{j:T[j] \neq T'[j]} |\alpha_{ij0}|^2 + |\alpha_{ij1}|^2 + \overline{\alpha_{ij0}} \alpha_{ij1} + \overline{\alpha_{ij1}} \alpha_{ij0} \right) \quad (23)$$

$$\geq 1 - 2 \sum_{j:T[j] \neq T'[j]} |\alpha_{ij0}|^2 + |\alpha_{ij1}|^2 \quad (24)$$

Since $U_{QRACM}(T')$ does not act on the purifying space, it commutes with tracing out the purifying

space, and partial trace can only decrease trace distance, we have that the trace distance between the states after applying QRACM to the two different tables is at most

$$\delta_i \leq \sqrt{2 \sum_{j: T[j] \neq T'[j]} |\alpha_{ij0}|^2 + |\alpha_{ij1}|^2} \quad (25)$$

where δ_i is defined in Equation (16).

We then let

$$m_j := \sum_{i=1}^d |\alpha_{ij0}|^2 + |\alpha_{ij1}|^2 \quad (26)$$

We know that $\sum_{j=0}^N m_j = d$. Thus, for any ℓ , there exist a set $\mathcal{J}$ of ℓ indices such that

$$\sum_{j \in \mathcal{J}} m_j \leq \frac{\ell d}{N}. \quad (27)$$

If we set T and T' to differ only on $\mathcal{J}$, we see that $\sum_{i=1}^d \delta_i^2 \leq 2 \sum_{j \in \mathcal{J}} m_j \leq \frac{2\ell d}{N}$, from which we see that

$$\sum_{i=1}^d \delta_i \leq d \sqrt{\frac{2\ell}{N}}. \quad (28)$$

□

We now argue by saying that since so little of the input state is concentrated on certain indices, we can replace any table T with a different table that differs on those indices, and this will be indistinguishable to the QRACM distillation-and-teleportation process. Since the process needs to handle all logical inputs, it must fail on that index.

Theorem C.2. *Suppose there is a QRACM state distillation-and-teleportation process, accessing tables of size $N = 2^n$ and making at most d calls to the physical QRACM. Then the minimal fidelity for this process is at most*

$$\frac{3}{4} + d \sqrt{\frac{2}{N}}. \quad (29)$$

Proof. Our distillation process is defined by some initial state ρ_1 , and d channels Φ_i , where we interleave access to the QRAM device with these channels. Then for each table T , we can then define states $\rho_i(T)$ such that

$$\rho_{i+1}(T) = \Phi_{i+1} \circ (I \otimes \mathcal{Q}(T)) \rho_i(T) \quad (30)$$

(using $\mathcal{Q}(T)$ as a more compact notation for $\mathcal{U}_{QRACM}(T)$, the unitary channel which performs

the QRACM access), with $\rho_1(T)$ defined to be ρ_1 for all tables T .

Let T be any table. Let $\mathcal{J}$ be the set of ℓ indices implied by Theorem C.1 for these states, and let T' be a table that differs from T in those ℓ indices. We will show by induction that

$$\|\rho_i(T') - \rho_i(T)\|_1 \leq \sum_{j=1}^i \delta_j \quad (31)$$

with δ_i as in Theorem C.1. This holds for $i = 1$ since all tables use the same starting state.

For induction, recall from the definition of the distillation process that

$$\rho_{i+1}(T) = \Phi_{i+1} \circ (I \otimes \mathcal{Q}(T)) \rho_i(T) \quad (32)$$

(similarly for T') where $\mathcal{Q}(T) = \mathcal{U}_{QRACM}(T)$ (a more compact notation), and Φ_i is the channel of any other computations performed during distillation between subsequent QRACM accesses. Since the channel Φ_{i+1} can only reduce the trace distance between then by the data processing inequality, we obtain:

$$\|\rho_{i+1}(T') - \rho_{i+1}(T)\|_1 \quad (33)$$

$$\leq \|(I \otimes \mathcal{Q}(T')) \rho_i(T') - (I \otimes \mathcal{Q}(T)) \rho_i(T)\|_1 \quad (34)$$

$$\leq \|((I \otimes \mathcal{Q}(T) - (I \otimes \mathcal{Q}(T'))) \rho_i(T)\|_1 \quad (35)$$

$$+ \|(I \otimes \mathcal{Q}(T'))(\rho_i(T') - \rho_i(T))\|_1 \quad (36)$$

$$\leq \delta_{i+1} + \sum_{j=1}^i \delta_j \quad (37)$$

using Theorem C.1.

We finally note that $\rho_d(T) = \rho_{\text{distill}}(T)$, the state input into the final teleportation process. This means the distance between $\rho_{\text{distill}}(T)$ and $\rho_{\text{distill}}(T')$ is at most $d \sqrt{\frac{2\ell}{N}}$ by Theorem C.1. Again by the data processing inequality, the final teleportation process cannot increase the distance. Denoting the full process as $\tilde{\mathcal{U}}_{QRACM}(T)$, we see that for any logical input ρ_L , we obtain

$$\|(\tilde{\mathcal{U}}_{QRACM}(T) - \tilde{\mathcal{U}}_{QRACM}(T')) \rho_L\|_1 \leq d \sqrt{\frac{2\ell}{N}} \quad (38)$$

However, let $\rho_L = |j\rangle\langle j| \otimes |0\rangle\langle 0|$ for some j with $T[j] \neq T'[j]$. Then we have for the *ideal* QRACM access,

$$\|(\mathcal{U}_{QRACM}(T) - \mathcal{U}_{QRACM}(T'))(\rho_L)\|_1 = 1 \quad (39)$$

But by the triangle inequality,

$$1 = \|(\mathcal{U}_{QRACM}(T) - \mathcal{U}_{QRACM}(T'))(\rho_L)\|_1 \quad (40)$$

$$\begin{aligned} &\leq \|(\mathcal{U}_{QRACM}(T) - \tilde{\mathcal{U}}_{QRACM}(T))\rho_L\|_1 \\ &\quad + \|(\mathcal{U}_{QRACM}(T') - \tilde{\mathcal{U}}_{QRACM}(T'))\rho_L\|_1 \\ &\quad + \|(\tilde{\mathcal{U}}_{QRACM}(T) - \tilde{\mathcal{U}}_{QRACM}(T'))\rho_L\|_1 \end{aligned} \quad (41)$$

$$\begin{aligned} 1 - d\sqrt{\frac{2\ell}{N}} &\leq \|(\mathcal{U}_{QRACM}(T) - \tilde{\mathcal{U}}_{QRACM}(T))\rho_L\|_1 \\ &\quad + \|(\mathcal{U}_{QRACM}(T') - \tilde{\mathcal{U}}_{QRACM}(T'))\rho_L\|_1 \end{aligned} \quad (42)$$

meaning (WLOG) that the distance between the realized QRACM channel and the ideal QRACM channel, for state ρ_L on access to table T or T' , is at least $\frac{1}{2} - d\sqrt{\frac{\ell}{2N}}$. Using known relations between trace distance and fidelity and setting $\ell = 1$ gives the result. $\square$